

maix.image

maix.image module, image related definition and functions

You can use `maix.image` to access this module with MaixPy

This module is generated from `MaixPy` and `MaixCDK`

1. Module

No module

2. Enum

2.1. Format

Image formats

item	describe
attention	for MaixPy firmware developers, update this enum will also need to update the fmt_size and fmt_names too !!!
values	FMT_RGB888 : RGBRGB...RGB, R at the lowest address FMT_BGR888 : BGRBGR...BGR, B at

item	describe
	the lowest address
	FMT_RGBA8888: RGBARGBA...RGBA, R at the lowest address
	FMT_BGRA8888: BGRABGRA...BGRA, B at the lowest address
	FMT_RGB565:
	FMT_BGR565:
	FMT_YUV422SP: YYY...UVUVUV...UVUV
	FMT_YUV422P: YYY...UUU...VVV
	FMT_YVU420SP: YYY...VUVUVU...VUVU, NV21
	FMT_YUV420SP: YYY...UVUVUV...UVUV, NV12
	FMT_YVU420P: YYY...VVV...UUU
	FMT_YUV420P: YYY...UUU...VVV
	FMT_GRAYSCALE:
	FMT_BGGR6: 6-bit Bayer format with a BGGR pattern.
	FMT_GBRG6: 6-bit Bayer format with a GBRG pattern.
	FMT_GRBG6: 6-bit Bayer format with a GRBG pattern.
	FMT_RGGB6: 6-bit Bayer format with a RGGB pattern.
	FMT_BGGR8: 8-bit Bayer format with a BGGR pattern.
	FMT_GBRG8: 8-bit Bayer format with a GBRG pattern.



item	describe
	FMT_GRBG8: 8-bit Bayer format with a GRBG pattern.
	FMT_RGGB8: 8-bit Bayer format with a RGGB pattern.
	FMT_BGGR10: 10-bit Bayer format with a BGGR pattern.
	FMT_GBRG10: 10-bit Bayer format with a GBRG pattern.
	FMT_GRBG10: 10-bit Bayer format with a GRBG pattern.
	FMT_RGGB10: 10-bit Bayer format with a RGGB pattern.
	FMT_BGGR12: 12-bit Bayer format with a BGGR pattern.
	FMT_GBRG12: 12-bit Bayer format with a GBRG pattern.
	FMT_GRBG12: 12-bit Bayer format with a GRBG pattern.
	FMT_RGGB12: 12-bit Bayer format with a RGGB pattern.
	FMT_UNCOMPRESSED_MAX:
	FMT_COMPRESSED_MIN:
	FMT_JPEG:
	FMT_PNG:
	FMT_COMPRESSED_MAX:
	FMT_INVALID: format not valid

C++ defination code:

```

1  enum Format
2      {
3      FMT_RGB888 = 0, //

```

```

4   RGBRGB...RGB, R at the lowest
5   address
6       FMT_BGR888,      //
7   BGRBGR...BGR, B at the lowest
8   address
9       FMT_RGBA8888,    //
10  RGBARGBA...RGBA, R at the lowest
11  address
12      FMT_BGRA8888,     //
13  BGRABGRA...BGRA, B at the lowest
14  address
15      FMT_RGB565,
16      FMT_BGR565,
17      FMT_YUV422SP, //
18  YYY...UVUVUV...UVUV
19      FMT_YUV422P,  //
20  YYY...UUU...VVV
21      FMT_YVU420SP, //
22  YYY...VUVUVU...VUVU, NV21
23      FMT_YUV420SP, //
24  YYY...UVUVUV...UVUV, NV12
25      FMT_YVU420P,  //
26  YYY...VVV...UUU
27      FMT_YUV420P,  //
28  YYY...UUU...VVV
29      FMT_GRAYSCALE,
30      FMT_BGGR6,      // 6-bit
31  Bayer format with a BGGR pattern.
32      FMT_GBRG6,      // 6-bit
33  Bayer format with a GBRG pattern.
34      FMT_GRBG6,      // 6-bit
35  Bayer format with a GRBG pattern.
36      FMT_RGGB6,      // 6-bit
37  Bayer format with a RGGB pattern.
38      FMT_BGGR8,      // 8-bit
39  Bayer format with a BGGR pattern.
40      FMT_GBRG8,      // 8-bit
41  Bayer format with a GBRG pattern.
      FMT_GRBG8,      // 8-bit
      Bayer format with a GRBG pattern.
      FMT_RGGB8,      // 8-bit
      Bayer format with a RGGB pattern.
      FMT_BGGR10,     // 10-bit

```

```

Bayer format with a BGGR pattern.
    FMT_GBRG10,        // 10-bit
Bayer format with a GBRG pattern.
    FMT_GRBG10,        // 10-bit
Bayer format with a GRBG pattern.
    FMT_RGGB10,        // 10-bit
Bayer format with a RGGG pattern.
    FMT_BGGR12,        // 12-bit
Bayer format with a BGGR pattern.
    FMT_GBRG12,        // 12-bit
Bayer format with a GBRG pattern.
    FMT_GRBG12,        // 12-bit
Bayer format with a GRBG pattern.
    FMT_RGGB12,        // 12-bit
Bayer format with a RGGG pattern.
    FMT_UNCOMPRESSED_MAX,

    // compressed format
below, not compressed should
define upper
    FMT_COMPRESSED_MIN,
    FMT_JPEG,
    FMT_PNG,
    FMT_COMPRESSED_MAX,

    FMT_INVALID = 0xFF //
format not valid
}

```

2.2. Fit

Object fit method

item	describe
values	FIT_NONE : no object fit, keep original FIT_FILL : width to new width, height to new height, may be stretch FIT_CONTAIN : keep aspect ratio, fill blank area with black color

item	describe
	FIT_COVER: keep aspect ratio, crop image to fit new size FIT_MAX:

C++ definition code:

```

1  enum Fit
2  {
3      FIT_NONE = -1, // no
4      object fit, keep original
5      FIT_FILL = 0,  // width
6      to new width, height to new
7      height, may be stretch
8      FIT_CONTAIN,  // keep
        aspect ratio, fill blank area
        with black color
        FIT_COVER,    // keep
        aspect ratio, crop image to fit
        new size
        FIT_MAX
    }

```

2.3. ResizeMethod

Resize method

item	describe
values	NEAREST: BILINEAR: BICUBIC: AREA: LANCZOS: HAMMING: RESIZE_METHOD_MAX:

C++ definition code:

```
1  enum ResizeMethod
2      {
3          NEAREST = 0,
4          BILINEAR,
5          BICUBIC,
6          AREA,
7          LANCZOS,
8          HAMMING,
9          RESIZE_METHOD_MAX
10     }
```

2.4. ApriltagFamilies

Family of apriltag

item	describe
values	TAG16H5: TAG25H7: TAG25H9: TAG36H10: TAG36H11: ARTOOLKIT:

C++ definition code:

```
1  enum ApriltagFamilies
2      {
3          TAG16H5    = 1,
4          TAG25H7    = 2,
5          TAG25H9    = 4,
6          TAG36H10   = 8,
7          TAG36H11   = 16,
8          ARTOOLKIT  = 32
9      }
```

```
}
```

2.5. TemplateMatch

Template match method

item	describe
values	SEARCH_EX : Exhaustive search SEARCH_DS : Diamond search

C++ definition code:

```
1 | enum TemplateMatch
2 | {
3 |     SEARCH_EX, // Exhaustive
4 |     search
5 |     SEARCH_DS, // Diamond
   |     search
   | }
```

2.6. CornerDetector

CornerDetector class

item	describe
values	CORNER_FAST : CORNER_AGAST :

C++ definition code:

```
1 | enum CornerDetector
2 | {
3 |     CORNER_FAST,
4 |     CORNER_AGAST
5 | }
```

```
}
```

2.7. EdgeDetector

EdgeDetector class

item	describe
values	EDGE_CANNY: EDGE_SIMPLE:

C++ definition code:

```
1  enum EdgeDetector
2      {
3          EDGE_CANNY,
4          EDGE_SIMPLE,
5      }
```

2.8. FlipDir

FlipDir

item	describe
values	X: Y: XY:

C++ definition code:

```
1  enum class FlipDir
2      {
3          X,
4          Y,
5          XY
6  }
```

```
}
```

2.9. QRCodeDecoderType

QRCode decode type class

item	describe
values	QRCODE_DECODER_TYPE_ZBAR: QRCODE_DECODER_TYPE_QUIRC:

C++ defination code:

```
1 | enum class QRCodeDecoderType {  
2 |     QRCODE_DECODER_TYPE_ZBAR,  
3 |     QRCODE_DECODER_TYPE_QUIRC  
4 | }
```

2.10. LineType

Line type class

item	describe
values	LINE_NORMAL: LINE_CROSS: LINE_T: LINE_L:

C++ defination code:

```
1 | enum class LineType {  
2 |     LINE_NORMAL,  
3 |     LINE_CROSS,  
4 |     LINE_T,  
5 |     LINE_L,  
6 | }
```

```
}
```

3. Variable

3.1. fmt_size

Image format size in bytes

item	description
attention	It's a copy of this variable in MaixPy, so change it in C++ (e.g. update var in hello function) will not take effect the var in MaixPy. So we add const for this var to avoid this mistake.
value	<pre>{ 3, 3, 4, 4, 2, 2, 2, 2, 1.5, 1.5, 1.5, 1.5, 1, // grayscale 0.75, // 6-bit Bayer format 0.75, // 6-bit Bayer format</pre>

item	description
	0.75, // 6-bit Bayer format 0.75, // 6-bit Bayer format 1, // 8-bit Bayer format 1, // 8-bit Bayer format 1, // 8-bit Bayer format 1, // 8-bit Bayer format 1.25, // 10-bit Bayer format 1.25, // 10-bit Bayer format 1.25, // 10-bit Bayer format 1.25, // 10-bit Bayer format 1.5, // 12-bit Bayer format 1.5, // 12-bit Bayer format 1.5, // 12-bit Bayer format 1.5, // 12-bit Bayer format 0, // uncomperreed_max 0, // compressed_min 1, // jpeg 1, // png 0, // compressed_max 0 // invalid }
readonly	True

C++ defination code:

```

1  const std::vector<float> fmt_size
2  = {
3      3,
4      3,
5      4,
6      4,
7      2,
8      2,

```

```
9      2,
10     2,
11     1.5,
12     1.5,
13     1.5,
14     1.5,
15     1, // grayscale
16     0.75, // 6-bit Bayer
17     format
18     0.75, // 6-bit Bayer
19     format
20     0.75, // 6-bit Bayer
21     format
22     0.75, // 6-bit Bayer
23     format
24     1, // 8-bit Bayer
25     format
26     1, // 8-bit Bayer
27     format
28     1, // 8-bit Bayer
29     format
30     1, // 8-bit Bayer
31     format
32     1.25, // 10-bit Bayer
33     format
34     1.25, // 10-bit Bayer
35     format
36     1.25, // 10-bit Bayer
37     format
        1.25, // 10-bit Bayer
        format
        1.5, // 12-bit Bayer
        format
        1.5, // 12-bit Bayer
        format
        1.5, // 12-bit Bayer
        format
        1.5, // 12-bit Bayer
        format
        0, // uncomperreed_max
        0, // compressed_min
        1, // jpeg
        1, // png
```

```
0, // compressed_max
0 // invalid
}
```

3.2. fmt_names

Image format string

item	description
value	{ "RGB888", "BGR888", "RGBA8888", "BGRA8888", "RGB565", "BGR565", "YUV422SP", "YUV422P", "YVU420SP", "YUV420SP", "YVU420P", "YUV420P", "GRAYSCALE", "BGGR6", "GBRG6", "GRBG6", "RG6B6", "BGGR8", "GBRG8", "GRBG8", "RG6B8", "BGGR10", "GBRG10",

item	description
	"GRBG10", "RG6B10", "BGGR12", "GBRG12", "GRBG12", "RG6B12", "UNCOMPRESSED_MAX", "COMPRESSED_MIN", "JPEG", "PNG", "COMPRESSED_MAX", "INVALID" }
readonly	True

C++ definition code:

```

1  const std::vector<std::string>
2  fmt_names = {
3      "RGB888",
4      "BGR888",
5      "RGBA8888",
6      "BGRA8888",
7      "RGB565",
8      "BGR565",
9      "YUV422SP",
10     "YUV422P",
11     "YVU420SP",
12     "YUV420SP",
13     "YVU420P",
14     "YUV420P",
15     "GRAYSCALE",
16     "BGGR6",
17     "GBRG6",
18     "GRBG6",
19

```

```

19         "RG6B6",
20         "BGGR8",
21         "GBRG8",
22         "GRBG8",
23         "RG6B8",
24         "BGGR10",
25         "GBRG10",
26         "GRBG10",
27         "RG6B10",
28         "BGGR12",
29         "GBRG12",
30         "GRBG12",
31         "RG6B12",
32         "UNCOMPRESSED_MAX",
33         "COMPRESSED_MIN",
34         "JPEG",
35         "PNG",
36         "COMPRESSED_MAX",
37         "INVALID"
    }

```

3.3. COLOR_WHITE

Predefined color white

item	description
value	<code>image::Color::from_rgb(255, 255, 255)</code>
readonly	True

C++ defination code:

```

1 | const image::Color COLOR_WHITE =
   | image::Color::from_rgb(255, 255,
   | 255)

```

3.4. COLOR_BLACK

Predefined color black

item	description
value	<code>image::Color::from_rgb(0, 0, 0)</code>
readonly	True

C++ definition code:

```
1 | const image::Color COLOR_BLACK =  
   | image::Color::from_rgb(0, 0, 0)
```

3.5. COLOR_RED

Predefined color red

item	description
value	<code>image::Color::from_rgb(255, 0, 0)</code>
readonly	True

C++ definition code:

```
1 | const image::Color COLOR_RED =  
   | image::Color::from_rgb(255, 0, 0)
```

3.6. COLOR_GREEN

Predefined color green

item	description
value	<code>image::Color::from_rgb(0, 255, 0)</code>
readonly	True

C++ definition code:

```
1 | const image::Color COLOR_GREEN =
   | image::Color::from_rgb(0, 255, 0)
```

3.7. COLOR_BLUE

Predefined color blue

item	description
value	<code>image::Color::from_rgb(0, 0, 255)</code>
readonly	True

C++ definition code:

```
1 | const image::Color COLOR_BLUE =
   | image::Color::from_rgb(0, 0, 255)
```

3.8. COLOR_YELLOW

Predefined color yellow

item	description
value	<code>image::Color::from_rgb(255, 255, 0)</code>

item	description
readonly	True
C++ definition code:	
<pre> 1 const image::Color COLOR_YELLOW = image::Color::from_rgb(255, 255, 0) </pre>	

3.9. COLOR_PURPLE

Predefined color purple

item	description
value	<code>image::Color::from_rgb(143, 0, 255)</code>
readonly	True
C++ definition code:	
<pre> 1 const image::Color COLOR_PURPLE = image::Color::from_rgb(143, 0, 255) </pre>	

3.10. COLOR_ORANGE

Predefined color orange

item	description
value	<code>image::Color::from_rgb(255, 127, 0)</code>

item	description
readonly	True
C++ definition code:	
<pre>1 const image::Color COLOR_ORANGE = image::Color::from_rgb(255, 127, 0)</pre>	

3.11. COLOR_GRAY

Predefined color gray

item	description
value	<code>image::Color::from_rgb(127, 127, 127)</code>
readonly	True
C++ definition code:	
<pre>1 const image::Color COLOR_GRAY = image::Color::from_rgb(127, 127, 127)</pre>	

4. Function

4.1. resize_map_pos

```
1 | def resize_map_pos(w_in: int, h_in:
   | int, w_out: int, h_out: int, fit:
   | Fit, x: int, y: int, w: int = -1, h:
   | int = -1) -> list[int]
```

map point position or rectangle position from one image size to another image size(resize)

item	description
param	int : h_out target image height fit : resize method, see <code>maix.image.Fit</code> x : original point x, or rectagle left-top point's x y : original point y, or rectagle left-top point's y w : original rectagle width, can be -1 if not use this arg, default -1. h : original rectagle height, can be -1 if not use this arg, default -1.
return	list type, [x, y] if map point, [x, y, w, h] if resize rectangle.

C++ defination code:

```
1 | std::vector<int>  
   | resize_map_pos(int w_in, int  
   | h_in, int w_out, int h_out,  
   | image::Fit fit, int x, int y, int  
   | w = -1, int h = -1)
```

4.2. `resize_map_pos_reverse`

```
1 | def resize_map_pos_reverse(w_in:  
   | int, h_in: int, w_out: int, h_out:  
   | int, fit: Fit, x: int, y: int, w:  
   | int = -1, h: int = -1) -> list[int]
```

reverse `resize_map_pos` method, when we call `image.resize` method resize image 'a' to image 'b', we want to know the original position on 'a' with a new point on 'b'

item	description
param	int : h_out image height after resized fit : resize method, see <code>maix.image.Fit</code> x : point on resized image x, or rectangle left-top point's x y : original point y, or rectangle left-top point's y w : original rectangle width, can be -1 if not use this arg, default -1. h : original rectangle height, can be -1 if not use this arg, default -1.
return	list type, [x, y] if map point, [x, y, w, h] if resize rectangle.

C++ definition code:

```
1 | std::vector<int>  
   | resize_map_pos_reverse(int w_in,  
   | int h_in, int w_out, int h_out,  
   | image::Fit fit, int x, int y, int  
   | w = -1, int h = -1)
```

4.3. load

```
1 | def load(path: str, format: Format =  
   | ...) -> Image
```

Load image from file, and convert to Image object

item	description
param	path : image file path format : read as this format, if not match, will convert to this format, by default is RGB888
return	Image object, if load failed, will return None(nullptr in C++), so you should care about it.

C++ definition code:

```
1 | image::Image *load(const char
   | *path, image::Format format =
   | image::Format::FMT_RGB888)
```

4.4. from_bytes

```
1 | def from_bytes(width: int, height:
   | int, format: Format, data:
   | maix.Bytes(bytes), copy: bool =
   | True) -> Image
```

Create image from bytes

item	description
param	width : image width height : image height format : image format data : image data, if data is None, will malloc memory for image data If the image is in jpeg format, data must be filled in.

item	description
	copy : if true and data is not None, will copy data to new buffer, else will use data directly. default is true to avoid memory leak. Use it carefully!!!
return	Image object

C++ definition code:

```
1 | image::Image *from_bytes(int
    | width, int height, image::Format
    | format, Bytes *data, bool copy =
    | true)
```

4.5. load_font

```
1 | def load_font(name: str, path: str,
    | size: int = 16) -> maix.err.Err
```

Load font from file

item	description
param	name : font name, used to identify font path : font file path, support ttf, ttc, otf size : font size, font height, by default is 16
return	error code, err::ERR_NONE is ok, other is error

C++ definition code:

```
1 | err::Err load_font(const  
   | std::string &name, const char  
   | *path, int size = 16)
```

4.6. set_default_font

Set default font, if not call this method, default is hershey_plain

item	description
param	name: font name, supported names can be get by fonts()
return	error code, err::ERR_NONE is ok, other is error

C++ definition code:

```
1 | err::Err set_default_font(const  
   | std::string &name)
```

4.7. fonts

```
1 | def fonts() -> list[str]
```

Get all loaded fonts

item	description
return	all loaded fonts, string list type

C++ definition code:

```
1 | std::vector<std::string> *fonts()
```

4.8. string_size

```
1 | def string_size(string: str, scale:  
    float = 1, thickness: int = 1, font:  
    str = '') -> Size
```

Get text rendered width and height

item	description
param	string : text content scale : font scale, by default(value is 1) thickness : text thickness(line width), by default(value is 1)
return	text rendered width and height, [width, height]

C++ definition code:

```
1 | image::Size  
    string_size(std::string string,  
    float scale = 1, int thickness =  
    1, const std::string &font = "")
```

4.9. cv2image

```
1 | def cv2image(array:  
    numpy.ndarray[numpy.uint8], bgr:  
    bool = True, copy: bool = True) ->  
    Image
```

OpenCV Mat(numpy array object) to Image object

item	description
param	array: numpy array object, must be a 3-dim or 2-dim continuous array with shape hwc or hw bgr: if set bgr, the return image will be marked as BGR888 or BGRA8888 format(only mark, not ensure return image is real BGR format), grayscale will ignore this arg. copy: if true, will alloc new buffer and copy data, else will directly use array's data buffer, default true. Use this arg carefully, when set to false, the array MUST keep alive until we don't use the return img of this func, or will cause program crash.
return	Image object

C++ definition code:

```
1 | image::Image
   | *cv2image(py::array_t<uint8_t,
   | py::array::c_style> array, bool
   | bgr = true, bool copy = true)
```

4.10. image2cv

```
1 | def image2cv(img: Image, ensure_bgr:
   | bool = True, copy: bool = True) ->
   | numpy.ndarray[numpy.uint8]
```

Image object to OpenCV Mat(numpy array object)

item	description
param	img: Image object, maix.image.Image type. ensure_bgr: auto convert to BGR888 or BGRA8888 if img format is not BGR or BGRA, if set to false, will not auto convert and directly use img's data, default true. If copy is false, ensure_bgr always be false. copy: Whether alloc new image and copy data or not, if ensure_bgr and img is not bgr or bgra format, always copy, if not copy, array object will directly use img's data buffer, will faster but change array will affect img's data, default true.
attention	take care of ensure_bgr and copy param.
return	numpy array object

C++ defination code:

```

1 | py::array_t<uint8_t,
   | py::array::c_style>
   | image2cv(image::Image *img, bool
   | ensure_bgr = true, bool copy =
   | true)

```

5. Class

5.1. Size

Image size type

C++ defination code:

```
1 | class Size
```

5.1.1. __init__

```
1 | def __init__(self, width: int = 0,  
    height: int = 0) -> None
```

Construct a new Size object

item	description
type	func
param	width : image width height : image height
static	False

C++ defination code:

```
1 | Size(int width = 0, int height =  
    0)
```

5.1.2. width

```
1 | def width(self, width: int = -1) ->
    int
```

width of size

item	description
type	func
param	width : set new width, if not set, only return current width
static	False

C++ defination code:

```
1 | int width(int width = -1)
```

5.1.3. height

```
1 | def height(self, height: int = -1) -
    > int
```

height of size

item	description
type	func
param	height : set new height, if not set, only return current height
static	False

C++ defination code:

```
1 | int height(int height = -1)
```

5.1.4. __getitem__

```
1 | def __getitem__(self, index: int) ->
   | int
```

Subscript operator

item	description
type	func
param	index: 0 for width, 1 for height
return	int& width or height
static	False

C++ defination code:

```
1 | int &operator[](int index)
```

5.1.5. __str__

```
1 | def __str__(self) -> str
```

to string

item	description
type	func
static	False

C++ defination code:

```
1 | std::string __str__()
```

5.2. Line

Line class

C++ definition code:

```
1 | class Line
```

5.2.1. __init__

```
1 | def __init__(self, x1: int, y1: int,
    x2: int, y2: int, magnitude: int =
    0, theta: int = 0, rho: int = 0) ->
    None
```

Line constructor

item	description
type	func
param	x1 : coordinate x1 of the straight line y1 : coordinate y1 of the straight line x2 : coordinate x2 of the straight line y2 : coordinate y2 of the straight line magnitude : magnitude of the straight line after Hough transformation theta : angle of the straight line after Hough transformation

item	description
	rho : p-value of the straight line after Hough transformation
static	False

C++ defination code:

```

1 | Line(int x1, int y1, int x2, int
   | y2, int magnitude = 0, int theta
   | = 0, int rho = 0)

```

5.2.2. __getitem__

```

1 | def __getitem__(self, index: int) ->
   | int

```

Subscript operator

item	description
type	func
param	index : [0] get x1 of line [1] get y1 of line [2] get x2 of line [3] get y2 of line [4] get length of line [5] get magnitude of the straight line after Hough transformation [6] get angle of the straight line after Hough transformation (0-179 degrees)

item	description
	[7] get p-value of the straight line after Hough transformation
return	int&
static	False

C++ defination code:

```
1 | int &__getitem__(int index)
```

5.2.3. x1

```
1 | def x1(self) -> int
```

get x1 of line

item	description
type	func
return	return x1 of the line, type is int
static	False

C++ defination code:

```
1 | int x1()
```

5.2.4. y1

```
1 | def y1(self) -> int
```

get y1 of line

item	description
type	func
return	return y1 of the line, type is int
static	False

C++ definition code:

```
1 | int y1()
```

5.2.5. x2

```
1 | def x2(self) -> int
```

get x2 of line

item	description
type	func
return	return x2 of the line, type is int
static	False

C++ definition code:

```
1 | int x2()
```

5.2.6. y2

```
1 | def y2(self) -> int
```

get y2 of line

item	description
type	func
return	return y2 of the line, type is int
static	False

C++ definition code:

```
1 | int y2()
```

5.2.7. length

```
1 | def length(self) -> int
```

get length of line

item	description
type	func
return	return length of the line, type is int
static	False

C++ definition code:

```
1 | int length()
```

5.2.8. magnitude

```
1 | def magnitude(self) -> int
```

get magnitude of the straight line after Hough transformation

item	description
type	func
return	return magnitude, type is int
static	False

C++ definition code:

```
1 | int magnitude()
```

5.2.9. theta

```
1 | def theta(self) -> int
```

get angle of the straight line after Hough transformation (0-179 degrees)

item	description
type	func
return	return angle, type is int
static	False

C++ definition code:

```
1 | int theta()
```

5.2.10. rho

```
1 | def rho(self) -> int
```

get p-value of the straight line after Hough transformation

item	description
type	func
return	return p-value, type is int
static	False

C++ definition code:

```
1 | int rho()
```

5.3. Rect

Rect class

C++ definition code:

```
1 | class Rect
```

5.3.1. __init__

```
1 | def __init__(self, corners:  
    list[list[int]], x: int, y: int, w:  
    int, h: int, magnitude: int = 0) ->  
    None
```

Rect constructor

item	description
type	func
param	corners: corners of rect x: coordinate x of the straight line y: coordinate y of the straight line w: coordinate w of the straight line h: coordinate h of the straight line magnitude: magnitude of the straight line after Hough transformation
static	False

C++ definition code:

```
1 | Rect(std::vector<std::vector<int>>
   | &corners, int x, int y, int w,
   | int h, int magnitude = 0)
```

5.3.2. __getitem__

```
1 | def __getitem__(self, index: int) ->
   | int
```

Subscript operator

item	description
type	func
param	index: [0] get x of rect [1] get y of rect [2] get w of rect

item	description
	[3] get h of rect [4] get magnitude of the straight line after Hough transformation
return	int&
static	False

C++ defination code:

```
1 | int &__getitem__(int index)
```

5.3.3. corners

```
1 | def corners(self) -> list[list[int]]
```

get corners of rect

item	description
type	func
return	return the coordinate of the rect.
static	False

C++ defination code:

```
1 | std::vector<std::vector<int>>
   | corners()
```

5.3.4. rect

```
1 | def rect(self) -> list[int]
```

get rectangle of rect

item	description
type	func
return	return the rectangle of the rect. format is {x, y, w, h}, type is std::vector
static	False

C++ defination code:

```
1 | std::vector<int> rect()
```

5.3.5. x

```
1 | def x(self) -> int
```

get x of rect

item	description
type	func
return	return x of the rect, type is int
static	False

C++ defination code:

```
1 | int x()
```

5.3.6. y

```
1 | def y(self) -> int
```

get y of rect

item	description
type	func
return	return y of the rect, type is int
static	False

C++ defination code:

```
1 | int y()
```

5.3.7. w

```
1 | def w(self) -> int
```

get w of rect

item	description
type	func
return	return w of the rect, type is int
static	False

C++ defination code:

```
1 | int w()
```

5.3.8. h

```
1 | def h(self) -> int
```

get h of rect

item	description
type	func
return	return h of the rect, type is int
static	False

C++ definition code:

```
1 | int h()
```

5.3.9. magnitude

```
1 | def magnitude(self) -> int
```

get magnitude of the straight line after Hough transformation

item	description
type	func
return	return magnitude, type is int
static	False

C++ definition code:

```
1 | int magnitude()
```

5.4. Circle

circle class

C++ definition code:

```
1 | class Circle
```

5.4.1. __init__

```
1 | def __init__(self, x: int, y: int,  
    r: int, magnitude: int) -> None
```

Circle constructor

item	description
type	func
param	x : coordinate x of the circle y : coordinate y of the circle r : coordinate r of the circle magnitude : coordinate y2 of the straight line
static	False

C++ definition code:

```
1 | Circle(int x, int y, int r, int  
    magnitude)
```

5.4.2. __getitem__

```
1 | def __getitem__(self, index: int) ->
    int
```

Subscript operator

item	description
type	func
param	index: [0] get x of circle [1] get y of circle [2] get r of circle [3] get magnitude of the circle after Hough transformation
return	int&
static	False

C++ defination code:

```
1 | int &__getitem__(int index)
```

5.4.3. x

```
1 | def x(self) -> int
```

get x of circle

item	description
type	func
return	return x of the circle, type is int
static	False

C++ definition code:

```
1 | int x()
```

5.4.4. y

```
1 | def y(self) -> int
```

get y of circle

item	description
type	func
return	return y of the circle, type is int
static	False

C++ definition code:

```
1 | int y()
```

5.4.5. r

```
1 | def r(self) -> int
```

get r of circle

item	description
type	func
return	return r of the circle, type is int
static	False

C++ definition code:

```
1 | int r()
```

5.4.6. magnitude

```
1 | def magnitude(self) -> int
```

get magnitude of the circle after Hough transformation

item	description
type	func
return	return magnitude, type is int
static	False

C++ definition code:

```
1 | int magnitude()
```

5.5. Blob

Blob class

C++ definition code:

```
1 | class Blob
```

5.5.1. __init__

```

1 | def __init__(self, rect: list[int],
    | corners: list[list[int]],
    | mini_corners: list[list[int]], cx:
    | float, cy: float, pixels: int,
    | rotation: float, code: int, count:
    | int, perimeter: int, roundness:
    | float, x_hist_bins: list[int],
    | y_hist_bins: list[int]) -> None

```

Blob constructor

item	description
type	func
param	rect: blob rect, type is std::vector corners: blob corners, type is std::vector<std::vector> mini_corners: blob mini_corners, type is std::vector<std::vector> cx: blob center x, type is float cy: blob center y, type is float pixels: blob pixels, type is int rotation: blob rotation, type is float code: blob code, type is int count: blob count, type is int perimeter: blob perimeter, type is int roundness: blob roundness, type is float x_hist_bins: blob x_hist_bins, type is std::vector y_hist_bins: blob y_hist_bins, type is std::vector
static	False

C++ definition code:

```
1 | Blob(std::vector<int> &rect,  
      std::vector<std::vector<int>>  
      &corners,  
      std::vector<std::vector<int>>  
      &mini_corners, float cx, float cy,  
      int pixels, float rotation, int  
      code, int count, int perimeter,  
      float roundness, std::vector<int>  
      &x_hist_bins, std::vector<int>  
      &y_hist_bins)
```

5.5.2. __getitem__

```
1 | def __getitem__(self, index: int) ->  
    int
```

Subscript operator

item	description
type	func
param	index: [0] Returns the blob' s bounding box x coordinate [1] Returns the blob' s bounding box y coordinate [2] Returns the blob' s bounding box w coordinate [3] Returns the blob' s bounding box h coordinate [4] Returns the number of pixels that are part of this blob [5] Returns the centroid x position of

item	description
	the blob [6] Returns the centroid y position of the blob
return	int& width or height
static	False

C++ defination code:

```
1 | int &__getitem__(int index)
```

5.5.3. corners

```
1 | def corners(self) -> list[list[int]]
```

get blob corners

item	description								
type	func								
return	Returns a list of 4 (x,y) tuples of the 4 corners of the object. (x0, y0)_____ (x1, y1)								

item	description
static	False

C++ definition code:

```
1 | std::vector<std::vector<int>>  
   | corners()
```

5.5.4. mini_corners

```
1 | def mini_corners(self) ->  
   | list[list[int]]
```

get blob mini corners

item	description
type	func
return	Returns a list of 4 (x,y) tuples of the 4 corners than bound the min area rectangle of the blob. (x0, y0)_____ (x1, y1)
static	False

C++ definition code:

```
1 | std::vector<std::vector<int>>
   | mini_corners()
```

5.5.5. rect

```
1 | def rect(self) -> list[int]
```

get blob rect

item	description						
type	func						
return	Returns the center coordinates and width and height of the rectangle. format is (x, y, w, h) w (x, y) _____				h		_____
static	False						

C++ defination code:

```
1 | std::vector<int> rect()
```

5.5.6. x

```
1 | def x(self) -> int
```

get blob x of the upper left coordinate

item	description
type	func
return	Returns the x coordinate of the upper left corner of the rectangle.
static	False

C++ defination code:

```
1 | int x()
```

5.5.7. y

```
1 | def y(self) -> int
```

get blob y of the upper left coordinate

item	description
type	func
return	Returns the y coordinate of the upper left corner of the rectangle.
static	False

C++ defination code:

```
1 | int y()
```

5.5.8. w

```
1 | def w(self) -> int
```

get blob width

item	description
type	func
return	Returns the blob' s bounding box w coordinate
static	False

C++ defination code:

```
1 | int w()
```

5.5.9. h

```
1 | def h(self) -> int
```

get blob height

item	description
type	func
return	Returns the blob' s bounding box h coordinate
static	False

C++ defination code:

```
1 | int h()
```

5.5.10. pixels

```
1 | def pixels(self) -> int
```

get blob pixels

item	description
type	func
return	Returns the number of pixels that are part of this blob.
static	False

C++ definition code:

```
1 | int pixels()
```

5.5.11. cx

```
1 | def cx(self) -> int
```

get blob center x

item	description
type	func
return	Returns the centroid x position of the blob
static	False

C++ definition code:

```
1 | int cx()
```

5.5.12. cy

```
1 | def cy(self) -> int
```

get blob center y

item	description
type	func
return	Returns the centroid y position of the blob
static	False

C++ definition code:

```
1 | int cy()
```

5.5.13. cxf

```
1 | def cxf(self) -> float
```

get blob center x

item	description
type	func

item	description
return	Returns the centroid x position of the blob
static	False
C++ definition code:	
<pre>1 float cxf()</pre>	

5.5.14. cyf

```
1 | def cyf(self) -> float
```

get blob center y

item	description
type	func
return	Returns the centroid y position of the blob
static	False
C++ definition code:	
<pre>1 float cyf()</pre>	

5.5.15. rotation

```
1 | def rotation(self) -> float
```

get blob rotation

item	description
type	func
return	Returns the rotation of the blob in radians (float). If the blob is like a pencil or pen this value will be unique for 0-180 degrees.
static	False

C++ definition code:

```
1 | float rotation()
```

5.5.16. rotation_rad

```
1 | def rotation_rad(self) -> float
```

get blob rotation_rad

item	description
type	func
return	Returns the rotation of the blob in radians
static	False

C++ definition code:

```
1 | float rotation_rad()
```

5.5.17. rotation_deg

```
1 | def rotation_deg(self) -> int
```

get blob rotation_deg

item	description
type	func
return	Returns the rotation of the blob in degrees.
static	False

C++ defination code:

```
1 | int rotation_deg()
```

5.5.18. code

```
1 | def code(self) -> int
```

get blob code

item	description
type	func
return	Returns a 32-bit binary number with a bit set in it for each color threshold that' s part of this blob
static	False

C++ defination code:

```
1 | int code()
```

5.5.19. count

```
1 | def count(self) -> int
```

get blob count

item	description
type	func
return	Returns the number of blobs merged into this blob.
static	False

C++ defination code:

```
1 | int count()
```

5.5.20. perimeter

```
1 | def perimeter(self) -> int
```

get blob merge_cnt

item	description
type	func
return	Returns the number of pixels on this blob' s perimeter.
static	False

C++ defination code:

```
1 | int perimeter()
```

5.5.21. roundness

```
1 | def roundness(self) -> float
```

get blob roundness

item	description
type	func
return	Returns a value between 0 and 1 representing how round the object is
static	False

C++ definition code:

```
1 | float roundness()
```

5.5.22. elongation

```
1 | def elongation(self) -> float
```

get blob elongation

item	description
type	func
returnReturns	a value between 0 and 1 representing how long (not round) the object is

item	description
static	False
C++ definition code:	
<pre>1 float elongation()</pre>	

5.5.23. area

```
1 | def area(self) -> int
```

get blob area

item	description
type	func
return	Returns the area of the bounding box around the blob
static	False
C++ definition code:	
<pre>1 int area()</pre>	

5.5.24. density

```
1 | def density(self) -> float
```

get blob density

item	description
type	func
return	Returns the density ratio of the blob
static	False

C++ definition code:

```
1 | float density()
```

5.5.25. extent

```
1 | def extent(self) -> float
```

Alias for blob.density()

item	description
type	func
return	Returns the density ratio of the blob
static	False

C++ definition code:

```
1 | float extent()
```

5.5.26. compactness

```
1 | def compactness(self) -> float
```

get blob compactness

item	description
type	func
return	Returns the compactness ratio of the blob
static	False

C++ definition code:

```
1 | float compactness()
```

5.5.27. solidity

```
1 | def solidity(self) -> float
```

get blob solidity

item	description
type	func
return	Returns the solidity ratio of the blob
static	False

C++ definition code:

```
1 | float solidity()
```

5.5.28. convexity

```
1 | def convexity(self) -> float
```

get blob convexity

item	description
type	func
return	Returns a value between 0 and 1 representing how convex the object is
static	False

C++ definition code:

```
1 | float convexity()
```

5.5.29. x_hist_bins

```
1 | def x_hist_bins(self) -> list[int]
```

get blob x_hist_bins

item	description
type	func
return	Returns the x_hist_bins of the blob
static	False

C++ definition code:

```
1 | std::vector<int> x_hist_bins()
```

5.5.30. y_hist_bins

```
1 | def y_hist_bins(self) -> list[int]
```

get blob y_hist_bins

item	description
type	func
return	Returns the y_hist_bins of the blob
static	False

C++ defination code:

```
1 | std::vector<int> y_hist_bins()
```

5.5.31. major_axis_line

```
1 | def major_axis_line(self) ->  
   | list[int]
```

get blob major_axis_line

item	description
type	func
return	Returns a line tuple (x1, y1, x2, y2) of the minor axis of the blob.
static	False

C++ defination code:

```
1 | std::vector<int>  
   | major_axis_line()
```

5.5.32. minor_axis_line

```
1 | def minor_axis_line(self) ->  
    list[int]
```

get blob minor_axis_line

item	description
type	func
return	Returns a line tuple (x1, y1, x2, y2) of the minor axis of the blob.
static	False

C++ definition code:

```
1 | std::vector<int>  
    minor_axis_line()
```

5.5.33. enclosing_circle

```
1 | def enclosing_circle(self) ->  
    list[int]
```

get blob enclosing_circle

item	description
type	func
return	Returns a circle tuple (x, y, r) of the circle that encloses the min area rectangle of a blob.

item	description
static	False
C++ definition code:	
<pre> 1 std::vector<int> enclosing_circle() </pre>	

5.5.34. enclosed_ellipse

```

1 | def enclosed_ellipse(self) ->
   | list[int]

```

get blob enclosed_ellipse

item	description
type	func
return	Returns an ellipse tuple (x, y, rx, ry, rotation) of the ellipse that fits inside of the min area rectangle of a blob.
static	False
C++ definition code:	
<pre> 1 std::vector<int> enclosed_ellipse() </pre>	

5.6. QRCode

QRCode class

C++ definition code:

```
1 | class QRCode
```

5.6.1. __init__

```
1 | def __init__(self, rect: list[int],  
  | corners: list[list[int]], payload:  
  | str, version: int, ecc_level: int,  
  | mask: int, data_type: int, eci: int)  
  | -> None
```

QRCode constructor

item	description
type	func
param	rect : rect of corners, type is std::vector corners : corners of QRCode payload : payload of the QRCode version : version of the QRCode ecc_level : ecc_level of the QRCode mask : mask of the QRCode data_type : data_type of the QRCode eci : eci of the QRCode
static	False

C++ definition code:

```
1 | QRCode(std::vector<int> &rect,  
  | std::vector<std::vector<int>>  
  | &corners, std::string &payload,
```

```
int version, int ecc_level, int  
mask, int data_type, int eci)
```

5.6.2. __getitem__

```
1 | def __getitem__(self, index: int) ->  
   int
```

Subscript operator

item	description
type	func
param	index: [0] Returns the qrcode' s bounding box x coordinate [1] Returns the qrcode' s bounding box y coordinate [2] Returns the qrcode' s bounding box w coordinate [3] Returns the qrcode' s bounding box h coordinate [4] Not support this index, try to use payload() method [5] Returns the version of qrcode [6] Returns the error correction level of qrcode [7] Returns the mask of qrcode [8] Returns the datatype of qrcode [9] Returns the eci of qrcode
return	int&
static	False

C++ definition code:

```
1 | int &__getitem__(int index)
```

5.6.3. corners

```
1 | def corners(self) -> list[list[int]]
```

get coordinate of QRCode

item	description
type	func
return	return the coordinate of the QRCode.
static	False

C++ definition code:

```
1 | std::vector<std::vector<int>>  
   | corners()
```

5.6.4. rect

```
1 | def rect(self) -> list[int]
```

get rectangle of QRCode

item	description
type	func
return	return the rectangle of the QRCode. format is {x, y, w, h}, type is

item	description
	std::vector
static	False
C++ defination code:	
<pre>1 std::vector<int> rect()</pre>	

5.6.5. x

```
1 | def x(self) -> int
```

get x of QRCode

item	description
type	func
return	return x of the QRCode, type is int
static	False
C++ defination code:	
<pre>1 int x()</pre>	

5.6.6. y

```
1 | def y(self) -> int
```

get y of QRCode

item	description
type	func
return	return y of the QRCode, type is int
static	False

C++ defination code:

```
1 | int y()
```

5.6.7. w

```
1 | def w(self) -> int
```

get w of QRCode

item	description
type	func
return	return w of the QRCode, type is int
static	False

C++ defination code:

```
1 | int w()
```

5.6.8. h

```
1 | def h(self) -> int
```

get h of QRCode

item	description
type	func
return	return h of the QRCode, type is int
static	False

C++ defination code:

```
1 | int h()
```

5.6.9. payload

```
1 | def payload(self) -> str
```

get QRCode payload

item	description
type	func
return	return area of the QRCode
static	False

C++ defination code:

```
1 | std::string payload()
```

5.6.10. version

```
1 | def version(self) -> int
```

get QRCode version

item	description
type	func
return	return version of the QRCode
static	False

C++ defination code:

```
1 | int version()
```

5.6.11. ecc_level

```
1 | def ecc_level(self) -> int
```

get QRCode error correction level

item	description
type	func
return	return error correction level of the QRCode
static	False

C++ defination code:

```
1 | int ecc_level()
```

5.6.12. mask

```
1 | def mask(self) -> int
```

get QRCode mask

item	description
type	func
return	return mask of the QRCode
static	False

C++ defination code:

```
1 | int mask()
```

5.6.13. data_type

```
1 | def data_type(self) -> int
```

get QRCode dataType

item	description
type	func
return	return mask of the QRCode
static	False

C++ defination code:

```
1 | int data_type()
```

5.6.14. eci

```
1 | def eci(self) -> int
```

get QRCode eci

item	description
type	func
return	return data of the QRCode
static	False

C++ defination code:

```
1 | int eci()
```

5.6.15. is_numeric

```
1 | def is_numeric(self) -> bool
```

check QRCode is numeric

item	description
type	func
return	return true if the result type of the QRCode is numeric
static	False

C++ defination code:

```
1 | bool is_numeric()
```

5.6.16. is_alphanumeric

```
1 | def is_alphanumeric(self) -> bool
```

check QRCode is alphanumeric

item	description
type	func
return	return true if the result type of the QRCode is alphanumeric
static	False

C++ defination code:

```
1 | bool is_alphanumeric()
```

5.6.17. is_binary

```
1 | def is_binary(self) -> bool
```

check QRCode is binary

item	description
type	func
return	return true if the result type of the QRCode is binary
static	False

C++ defination code:

```
1 | bool is_binary()
```

5.6.18. is_kanji

```
1 | def is_kanji(self) -> bool
```

check QRCode is kanji

item	description
type	func
return	return true if the result type of the QRCode is kanji
static	False

C++ defination code:

```
1 | bool is_kanji()
```

5.7. AprilTag

AprilTag class

C++ defination code:

```
1 | class AprilTag
```

5.7.1. __init__

```
1 | def __init__(self, rect: list[int],  
    corners: list[list[int]], id: int,  
    famliy: int, centroid_x: float,  
    centroid_y: float, rotation: float,  
    decision_margin: float, hamming:  
    int, goodness: float, x_translation:  
    float, y_translation: float,  
    z_translation: float, x_rotation:  
    float, y_rotation: float,
```

```
z_rotation: float) -> None
```

AprilTag constructor

item	description
type	func
param	rect: Includes the top-left corner and the width and height of the rectangle. format is {x, y, w, h}, type is std::vector corners: Includes the four corners of the rectangle. format is {{x0, y0}, {x1, y1}, {x2, y2}, {x3, y3}}, type is std::vector<std::vector> id: The id of the AprilTag family: The family of the AprilTag centroid_x: The x coordinate of the center of the AprilTag centroid_y: The y coordinate of the center of the AprilTag rotation: The rotation of the AprilTag decision_margin: The decision_margin of the AprilTag hamming: The hamming of the AprilTag goodness: The goodness of the AprilTag x_translation: The x_translation of the AprilTag y_translation: The y_translation of the AprilTag z_translation: The z_translation of

item	description
	the AprilTag x_rotation: The x_rotation of the AprilTag y_rotation: The y_rotation of the AprilTag z_rotation: The z_rotation of the AprilTag
static	False

C++ defination code:

```

1 | AprilTag(std::vector<int> &rect,
   | std::vector<std::vector<int>>
   | &corners, int id, int famliy,
   | float centroid_x, float
   | centroid_y, float rotation, float
   | decision_margin, int hamming,
   | float goodness, float
   | x_translation, float
   | y_translation, float
   | z_translation, float x_rotation,
   | float y_rotation, float
   | z_rotation)

```

5.7.2. __getitem__

```

1 | def __getitem__(self, index: int) ->
   | int

```

Subscript operator

item	description
type	func

item	description
param	index: [0] Returns the apriltag' s bounding box x coordinate [1] Returns the apriltag' s bounding box y coordinate [2] Returns the apriltag' s bounding box w coordinate [3] Returns the apriltag' s bounding box h coordinate [4] Returns the apriltag' s id [5] Returns the apriltag' s family [6] Not support [7] Not support [8] Not support [9] Not support [10] Returns the apriltag' s hamming [11] Not support [12] Not support [13] Not support [14] Not support [15] Not support [16] Not support [17] Not support
return	int&
static	False
C++ defination code: <pre> 1 int &__getitem__(int index) </pre>	

5.7.3. corners

```
1 | def corners(self) -> list[list[int]]
```

get coordinate of AprilTag

item	description
type	func
return	return the coordinate of the AprilTag.
static	False

C++ defination code:

```
1 | std::vector<std::vector<int>>  
   | corners()
```

5.7.4. rect

```
1 | def rect(self) -> list[int]
```

get rectangle of AprilTag

item	description
type	func
return	return the rectangle of the AprilTag. format is {x, y, w, h}, type is std::vector
static	False

C++ defination code:

```
1 | std::vector<int> rect()
```

5.7.5. x

```
1 | def x(self) -> int
```

get x of AprilTag

item	description
type	func
return	return x of the AprilTag, type is int
static	False

C++ definition code:

```
1 | int x()
```

5.7.6. y

```
1 | def y(self) -> int
```

get y of AprilTag

item	description
type	func
return	return y of the AprilTag, type is int
static	False

C++ definition code:

```
1 | int y()
```

5.7.7. w

```
1 | def w(self) -> int
```

get w of AprilTag

item	description
type	func
return	return w of the AprilTag, type is int
static	False

C++ defination code:

```
1 | int w()
```

5.7.8. h

```
1 | def h(self) -> int
```

get h of AprilTag

item	description
type	func
return	return h of the AprilTag, type is int
static	False

C++ defination code:

```
1 | int h()
```

5.7.9. id

```
1 | def id(self) -> int
```

get id of AprilTag

item	description
type	func
return	return id of the AprilTag, type is int
static	False

C++ definition code:

```
1 | int id()
```

5.7.10. family

```
1 | def family(self) -> int
```

get family of AprilTag

item	description
type	func
return	return family of the AprilTag, type is int
static	False

C++ definition code:

```
1 | int family()
```

5.7.11. cx

```
1 | def cx(self) -> int
```

get cx of AprilTag

item	description
type	func
return	return cx of the AprilTag, type is int
static	False

C++ definition code:

```
1 | int cx()
```

5.7.12. cxf

```
1 | def cxf(self) -> float
```

get cxf of AprilTag

item	description
type	func
return	return cxf of the AprilTag, type is float
static	False

C++ definition code:

```
1 | float cxf()
```

5.7.13. cy

```
1 | def cy(self) -> int
```

get cy of AprilTag

item	description
type	func
return	return cy of the AprilTag, type is int
static	False

C++ definition code:

```
1 | int cy()
```

5.7.14. cyf

```
1 | def cyf(self) -> float
```

get cyf of AprilTag

item	description
type	func
return	return cyf of the AprilTag, type is float
static	False

C++ definition code:

```
1 | float cyf()
```

5.7.15. rotation

```
1 | def rotation(self) -> float
```

get rotation of AprilTag

item	description
type	func
return	return rotation of the AprilTag, type is float
static	False

C++ definition code:

```
1 | float rotation()
```

5.7.16. decision_margin

```
1 | def decision_margin(self) -> float
```

Get decision_margin of AprilTag

item	description
type	func

item	description
return	Returns the quality of the apriltag match (0.0 - 1.0) where 1.0 is the best.
static	False

C++ definition code:

```
1 | float decision_margin()
```

5.7.17. hamming

```
1 | def hamming(self) -> int
```

get hamming of AprilTag

item	description
type	func
return	Returns the number of accepted bit errors for this tag. return 0, means 0 bit errors will be accepted. 1 is TAG25H7, means up to 1 bit error may be accepted 2 is TAG25H9, means up to 3 bit errors may be accepted 3 is TAG36H10, means up to 3 bit errors may be accepted 4 is TAG36H11, means up to 4 bit errors may be accepted

item	description
	5 is ARTOOLKIT, means 0 bit errors will be accepted
static	False
C++ defination code:	
<pre>1 int hamming()</pre>	

5.7.18. goodness

```
1 | def goodness(self) -> float
```

get goodness of AprilTag

item	description
type	func
return	return goodness of the AprilTag, type is float Note: This value is always 0.0 for now.
static	False
C++ defination code:	
<pre>1 float goodness()</pre>	

5.7.19. x_translation

```
1 | def x_translation(self) -> float
```

get x_translation of AprilTag

item	description
type	func
return	return x_translation of the AprilTag, type is float
static	False

C++ definition code:

```
1 | float x_translation()
```

5.7.20. y_translation

```
1 | def y_translation(self) -> float
```

get y_translation of AprilTag

item	description
type	func
return	return y_translation of the AprilTag, type is float
static	False

C++ definition code:

```
1 | float y_translation()
```

5.7.21. z_translation

```
1 | def z_translation(self) -> float
```

get z_translation of AprilTag

item	description
type	func
return	return z_translation of the AprilTag, type is float
static	False

C++ defination code:

```
1 | float z_translation()
```

5.7.22. x_rotation

```
1 | def x_rotation(self) -> float
```

get x_rotation of AprilTag

item	description
type	func
return	return x_rotation of the AprilTag, type is float
static	False

C++ defination code:

```
1 | float x_rotation()
```

5.7.23. y_rotation

```
1 | def y_rotation(self) -> float
```

get y_rotation of AprilTag

item	description
type	func
return	return y_rotation of the AprilTag, type is float
static	False

C++ defination code:

```
1 | float y_rotation()
```

5.7.24. z_rotation

```
1 | def z_rotation(self) -> float
```

get z_rotation of AprilTag

item	description
type	func
return	return z_rotation of the AprilTag, type is float
static	False

C++ defination code:

```
1 | float z_rotation()
```

5.8. DataMatrix

DataMatrix class

C++ definition code:

```
1 | class DataMatrix
```

5.8.1. __init__

```
1 | def __init__(self, rect: list[int],  
    corners: list[list[int]], payload:  
    str, rotation: float, rows: int,  
    columns: int, capacity: int,  
    padding: int) -> None
```

DataMatrix constructor

item	description
type	func
param	rect: Includes the top-left corner and the width and height of the rectangle. format is {x, y, w, h}, type is std::vector corners: Includes the four corners of the rectangle. format is {{x0, y0}, {x1, y1}, {x2, y2}, {x3, y3}}, type is std::vector<std::vector> payload: The payload of the DataMatrix

item	description
	rotation: The rotation of the DataMatrix rows: The rows of the DataMatrix columns: The columns of the DataMatrix capacity: The capacity of the DataMatrix padding: The padding of the DataMatrix
static	False

C++ defination code:

```
1 | DataMatrix(std::vector<int>
    &rect,
    std::vector<std::vector<int>>
    &corners, std::string &payload,
    float rotation, int rows, int
    columns, int capacity, int
    padding)
```

5.8.2. __getitem__

```
1 | def __getitem__(self, index: int) ->
    int
```

Subscript operator

item	description
type	func

item	description
param	index: [0] get x of DataMatrix [1] get y of DataMatrix [2] get w of DataMatrix [3] get h of DataMatrix [4] Not support this index, try to use payload() method [5] Not support this index, try to use rotation() method [6] get rows of DataMatrix [7] get columns of DataMatrix [8] get capacity of DataMatrix [9] get padding of DataMatrix
return	int&
static	False

C++ defination code:

```
1 | int &__getitem__(int index)
```

5.8.3. corners

```
1 | def corners(self) -> list[list[int]]
```

get coordinate of DataMatrix

item	description
type	func
return	return the coordinate of the DataMatrix.

item	description
static	False
C++ definition code:	
<pre>1 std::vector<std::vector<int>> corners()</pre>	

5.8.4. rect

```
1 | def rect(self) -> list[int]
```

get rectangle of DataMatrix

item	description
type	func
return	return the rectangle of the DataMatrix. format is {x, y, w, h}, type is std::vector
static	False
C++ definition code:	
<pre>1 std::vector<int> rect()</pre>	

5.8.5. x

```
1 | def x(self) -> int
```

get x of DataMatrix

item	description
type	func
return	return x of the DataMatrix, type is int
static	False

C++ definition code:

```
1 | int x()
```

5.8.6. y

```
1 | def y(self) -> int
```

get y of DataMatrix

item	description
type	func
return	return y of the DataMatrix, type is int
static	False

C++ definition code:

```
1 | int y()
```

5.8.7. w

```
1 | def w(self) -> int
```

get w of DataMatrix

item	description
type	func
return	return w of the DataMatrix, type is int
static	False

C++ definition code:

```
1 | int w()
```

5.8.8. h

```
1 | def h(self) -> int
```

get h of DataMatrix

item	description
type	func
return	return h of the DataMatrix, type is int
static	False

C++ definition code:

```
1 | int h()
```

5.8.9. payload

```
1 | def payload(self) -> str
```

get payload of DataMatrix

item	description
type	func
return	return payload of the DataMatrix, type is std::string
static	False

C++ definition code:

```
1 | std::string payload()
```

5.8.10. rotation

```
1 | def rotation(self) -> float
```

get rotation of DataMatrix

item	description
type	func
return	return rotation of the DataMatrix, type is float
static	False

C++ definition code:

```
1 | float rotation()
```

5.8.11. rows

```
1 | def rows(self) -> int
```

get rows of DataMatrix

item	description
type	func
return	return rows of the DataMatrix, type is int
static	False

C++ definition code:

```
1 | int rows()
```

5.8.12. columns

```
1 | def columns(self) -> int
```

get columns of DataMatrix

item	description
type	func
return	return columns of the DataMatrix, type is int
static	False

C++ definition code:

```
1 | int columns()
```

5.8.13. capacity

```
1 | def capacity(self) -> int
```

get capacity of DataMatrix

item	description
type	func
return	returns how many characters could fit in this data matrix, type is int
static	False

C++ defination code:

```
1 | int capacity()
```

5.8.14. padding

```
1 | def padding(self) -> int
```

get padding of DataMatrix

item	description
type	func
return	returns how many unused characters are in this data matrix, type is int
static	False

C++ defination code:

```
1 | int padding()
```

5.9. BarCode

BarCode class

C++ definition code:

```
1 | class BarCode
```

5.9.1. __init__

```
1 | def __init__(self, rect: list[int],  
    corners: list[list[int]], payload:  
    str, type: int, rotation: float,  
    quality: int) -> None
```

BarCode constructor

item	description
type	func
param	rect: Includes the top-left corner and the width and height of the rectangle. format is {x, y, w, h}, type is std::vector corners: Includes the four corners of the rectangle. format is {{x0, y0}, {x1, y1}, {x2, y2}, {x3, y3}}, type is std::vector<std::vector> payload: The payload of the BarCode type: The type of the BarCode rotation: The rotation of the BarCode quality: The quality of the BarCode

item	description
static	False
C++ definition code:	
<pre> 1 Barcode(std::vector<int> &rect, std::vector<std::vector<int>> &corners, std::string &payload, int type, float rotation, int quality) </pre>	

5.9.2. __getitem__

```

1 | def __getitem__(self, index: int) ->
    | int

```

Subscript operator

item	description
type	func
param	index: [0] get x of Barcode [1] get y of Barcode [2] get w of Barcode [3] get h of Barcode [4] Not support this index, try to use payload() method [5] get type of Barcode [6] Not support this index, try to use rotation() method [7] get quality of Barcode
return	int&

item	description
static	False
C++ definition code:	
<pre>1 int &__getitem__(int index)</pre>	

5.9.3. corners

```
1 | def corners(self) -> list[list[int]]
```

get coordinate of BarCode

item	description
type	func
return	return the coordinate of the BarCode.
static	False
C++ definition code:	
<pre>1 std::vector<std::vector<int>> corners()</pre>	

5.9.4. rect

```
1 | def rect(self) -> list[int]
```

get rectangle of BarCode

item	description
type	func
return	return the rectangle of the BarCode. format is {x, y, w, h}, type is std::vector
static	False

C++ definition code:

```
1 | std::vector<int> rect()
```

5.9.5. x

```
1 | def x(self) -> int
```

get x of BarCode

item	description
type	func
return	return x of the BarCode, type is int
static	False

C++ definition code:

```
1 | int x()
```

5.9.6. y

```
1 | def y(self) -> int
```

get y of BarCode

item	description
type	func
return	return y of the BarCode, type is int
static	False

C++ definition code:

```
1 | int y()
```

5.9.7. w

```
1 | def w(self) -> int
```

get w of BarCode

item	description
type	func
return	return w of the BarCode, type is int
static	False

C++ definition code:

```
1 | int w()
```

5.9.8. h

```
1 | def h(self) -> int
```

get h of BarCode

item	description
type	func
return	return h of the BarCode, type is int
static	False

C++ defination code:

```
1 | int h()
```

5.9.9. payload

```
1 | def payload(self) -> str
```

get payload of BarCode

item	description
type	func
return	return payload of the BarCode, type is std::string
static	False

C++ defination code:

```
1 | std::string payload()
```

5.9.10. type

```
1 | def type(self) -> int
```

get type of BarCode

item	description
type	func
return	return type of the BarCode, type is int
static	False

C++ defination code:

```
1 | int type()
```

5.9.11. rotation

```
1 | def rotation(self) -> float
```

get rotation of BarCode

item	description
type	func
return	return rotation of the BarCode, type is float. FIXME: always return 0.0
static	False

C++ defination code:

```
1 | float rotation()
```

5.9.12. quality

```
1 | def quality(self) -> int
```

get quality of BarCode

item	description
type	func
return	return quality of the BarCode, type is int
static	False

C++ defination code:

```
1 | int quality()
```

5.10. Statistics

Statistics class

C++ defination code:

```
1 | class Statistics
```

5.10.1. __init__

```
1 | def __init__(self, format: Format,  
    l_statistics: list[int],  
    a_statistics: list[int],  
    b_statistics: list[int]) -> None
```

Statistics constructor

item	description
type	func
param	format: The statistics source image format l_statistics: The statistics of the L channel. format is {mean, median, mode, std_dev, min, max, lq, uq}, type is std::vector a_statistics: The statistics of the A channel. format is {mean, median, mode, std_dev, min, max, lq, uq}, type is std::vector b_statistics: The statistics of the B channel. format is {mean, median, mode, std_dev, min, max, lq, uq}, type is std::vector
static	False

C++ defination code:

```
1 | Statistics(image::Format format,
   | std::vector<int> &l_statistics,
   | std::vector<int> &a_statistics,
   | std::vector<int> &b_statistics)
```

5.10.2. __getitem__

```
1 | def __getitem__(self, index: int) ->
   | int
```

Subscript operator

item	description
type	func
param	index: array index
return	int&
static	False

C++ definition code:

```
1 | int &__getitem__(int index)
```

5.10.3. format

```
1 | def format(self) -> Format
```

get format of Statistics source image

item	description
type	func
return	return format of the Statistics source image, type is image::Format
static	False

C++ definition code:

```
1 | image::Format format()
```

5.10.4. l_mean

```
1 | def l_mean(self) -> int
```

get L channel mean

item	description
type	func
return	return L channel mean, type is int
static	False

C++ defination code:

```
1 | int l_mean()
```

5.10.5. l_median

```
1 | def l_median(self) -> int
```

get L channel median

item	description
type	func
return	return L channel median, type is int
static	False

C++ defination code:

```
1 | int l_median()
```

5.10.6. l_mode

```
1 | def l_mode(self) -> int
```

get L channel mode

item	description
type	func
return	return L channel mode, type is int
static	False

C++ defination code:

```
1 | int l_mode()
```

5.10.7. l_std_dev

```
1 | def l_std_dev(self) -> int
```

get L channel std_dev

item	description
type	func
return	return L channel std_dev, type is int
static	False

C++ defination code:

```
1 | int l_std_dev()
```

5.10.8. l_min

```
1 | def l_min(self) -> int
```

get L channel min

item	description
type	func
return	return L channel min, type is int
static	False

C++ defination code:

```
1 | int l_min()
```

5.10.9. l_max

```
1 | def l_max(self) -> int
```

get L channel max

item	description
type	func
return	return L channel max, type is int
static	False

C++ defination code:

```
1 | int l_max()
```

5.10.10. l_lq

```
1 | def l_lq(self) -> int
```

get L channel lq

item	description
type	func
return	return L channel lq, type is int
static	False

C++ defination code:

```
1 | int l_lq()
```

5.10.11. l_uq

```
1 | def l_uq(self) -> int
```

get L channel uq

item	description
type	func
return	return L channel uq, type is int
static	False

C++ defination code:

```
1 | int l_uq()
```

5.10.12. a_mean

```
1 | def a_mean(self) -> int
```

get A channel mean

item	description
type	func
return	return A channel mean, type is int
static	False

C++ defination code:

```
1 | int a_mean()
```

5.10.13. a_median

```
1 | def a_median(self) -> int
```

get A channea median

item	description
type	func
return	return A channel median, type is int
static	False

C++ defination code:

```
1 | int a_median()
```

5.10.14. a_mode

```
1 | def a_mode(self) -> int
```

get A channel mode

item	description
type	func
return	return A channel mode, type is int
static	False

C++ defination code:

```
1 | int a_mode()
```

5.10.15. a_std_dev

```
1 | def a_std_dev(self) -> int
```

get A channel std_dev

item	description
type	func
return	return A channel std_dev, type is int
static	False

C++ defination code:

```
1 | int a_std_dev()
```

5.10.16. a_min

```
1 | def a_min(self) -> int
```

get A channel min

item	description
type	func
return	return A channel min, type is int
static	False

C++ defination code:

```
1 | int a_min()
```

5.10.17. a_max

```
1 | def a_max(self) -> int
```

get A channel max

item	description
type	func
return	return A channel max, type is int
static	False

C++ defination code:

```
1 | int a_max()
```

5.10.18. a_lq

```
1 | def a_lq(self) -> int
```

get A channel lq

item	description
type	func
return	return A channel lq, type is int
static	False

C++ defination code:

```
1 | int a_lq()
```

5.10.19. a_uq

```
1 | def a_uq(self) -> int
```

get A channel uq

item	description
type	func
return	return A channel uq, type is int
static	False

C++ defination code:

```
1 | int a_uq()
```

5.10.20. b_mean

```
1 | def b_mean(self) -> int
```

get B channel mean

item	description
type	func
return	return B channel mean, type is int
static	False

C++ definition code:

```
1 | int b_mean()
```

5.10.21. b_median

```
1 | def b_median(self) -> int
```

get B channel median

item	description
type	func
return	return B channel median, type is int
static	False

C++ definition code:

```
1 | int b_median()
```

5.10.22. b_mode

```
1 | def b_mode(self) -> int
```

get B channel mode

item	description
type	func
return	return B channel mode, type is int
static	False

C++ defination code:

```
1 | int b_mode()
```

5.10.23. b_std_dev

```
1 | def b_std_dev(self) -> int
```

get B channel std_dev

item	description
type	func
return	return B channel std_dev, type is int
static	False

C++ defination code:

```
1 | int b_std_dev()
```

5.10.24. b_min

```
1 | def b_min(self) -> int
```

get B channel min

item	description
type	func
return	return B channel min, type is int
static	False

C++ definition code:

```
1 | int b_min()
```

5.10.25. b_max

```
1 | def b_max(self) -> int
```

get B channel max

item	description
type	func
return	return B channel max, type is int
static	False

C++ definition code:

```
1 | int b_max()
```

5.10.26. b_lq

```
1 | def b_lq(self) -> int
```

get B channel lq

item	description
type	func
return	return B channel lq, type is int
static	False

C++ defination code:

```
1 | int b_lq()
```

5.10.27. b_uq

```
1 | def b_uq(self) -> int
```

get B channel uq

item	description
type	func
return	return B channel uq, type is int
static	False

C++ defination code:

```
1 | int b_uq()
```

5.11. Displacement

Displacement class

C++ definition code:

```
1 | class Displacement
```

5.11.1. __init__

```
1 | def __init__(self, x_translation:  
    float, y_translation: float,  
    rotation: float, scale: float,  
    response: float) -> None
```

Displacement constructor

item	description
type	func
param	x_translation : The x_translation of the Displacement y_translation : The y_translation of the Displacement rotation : The rotation of the Displacement scale : The scale of the Displacement response : The response of the Displacement
static	False

C++ definition code:

```
1 | Displacement(float x_translation,  
    float y_translation, float  
    rotation, float scale, float
```

```
response)
```

5.11.2. `__getitem__`

```
1 | def __getitem__(self, index: int) ->
   | int
```

Subscript operator

item	description
type	func
param	index: array index
return	int&
static	False

C++ definition code:

```
1 | int &__getitem__(int index)
```

5.11.3. `x_translation`

```
1 | def x_translation(self) -> float
```

get `x_translation` of Displacement

item	description
type	func
return	return <code>x_translation</code> of the Displacement, type is float

item	description
static	False
C++ definition code:	
<pre>1 float x_translation()</pre>	

5.11.4. y_translation

```
1 | def y_translation(self) -> float
```

get y_translation of Displacement

item	description
type	func
return	return y_translation of the Displacement, type is float
static	False
C++ definition code:	
<pre>1 float y_translation()</pre>	

5.11.5. rotation

```
1 | def rotation(self) -> float
```

get rotation of Displacement

item	description
type	func
return	return rotation of the Displacement, type is float
static	False

C++ definition code:

```
1 | float rotation()
```

5.11.6. scale

```
1 | def scale(self) -> float
```

get scale of Displacement

item	description
type	func
return	return scale of the Displacement, type is float
static	False

C++ definition code:

```
1 | float scale()
```

5.11.7. response

```
1 | def response(self) -> float
```

get response of Displacement

item	description
type	func
return	return response of the Displacement, type is float
static	False

C++ defination code:

```
1 | float response()
```

5.12. Percentile

Percentile class

C++ defination code:

```
1 | class Percentile
```

5.12.1. __init__

```
1 | def __init__(self, l_value: int,  
  a_value: int = 0, b_value: int = 0)  
  -> None
```

Percentile constructor

item	description
type	func

item	description
param	l_value: for grayscale image, it is grayscale percentile value (between 0 and 255). for rgb888 image, it is l channel percentile value of lab (between 0 and 100). a_value: for rgb888 image, it is a channel percentile value of lab format(between -128 and 127). b_value: for rgb888 image, it is b channel percentile value of lab format(between -128 and 127).
static	False

C++ defination code:

```
1 | Percentile(int l_value, int
   | a_value = 0, int b_value = 0)
```

5.12.2. __getitem__

```
1 | def __getitem__(self, index: int) ->
   | int
```

Subscript operator

item	description
type	func
static	False

C++ definition code:

```
1 | int &__getitem__(int index)
```

5.12.3. value

```
1 | def value(self) -> int
```

Return the grayscale percentile value (between 0 and 255).

item	description
type	func
return	returns grayscale percentile value
static	False

C++ definition code:

```
1 | int value()
```

5.12.4. l_value

```
1 | def l_value(self) -> int
```

Return the l channel percentile value of lab format (between 0 and 100).

item	description
type	func

item	description
return	returns l channel percentile value
static	False

C++ definition code:

```
1 | int l_value()
```

5.12.5. a_value

```
1 | def a_value(self) -> int
```

Return the a channel percentile value of lab format (between -128 and 127).

item	description
type	func
return	returns a channel percentile value
static	False

C++ definition code:

```
1 | int a_value()
```

5.12.6. b_value

```
1 | def b_value(self) -> int
```

Return the b channel percentile value of lab format (between -128 and 127).

item	description
type	func
return	returns b channel percentile value
static	False

C++ definition code:

```
1 | int b_value()
```

5.13. Threshold

Threshold class

C++ definition code:

```
1 | class Threshold
```

5.13.1. __init__

```
1 | def __init__(self, l_value: int,
    a_value: int = 0, b_value: int = 0)
    -> None
```

Threshold constructor

item	description
type	func
param	l_value : for grayscale image, it is grayscale threshold value (between 0 and 255).

item	description
	for rgb888 image, it is l channel threshold value of lab (between 0 and 100). a_value: for rgb888 image, it is a channel threshold value of lab format(between -128 and 127). b_value: for rgb888 image, it is b channel threshold value of lab format(between -128 and 127).
static	False

C++ definition code:

```
1 | Threshold(int l_value, int
   | a_value = 0, int b_value = 0)
```

5.13.2. __getitem__

```
1 | def __getitem__(self, index: int) ->
   | int
```

Subscript operator

item	description
type	func
static	False

C++ definition code:

```
1 | int &__getitem__(int index)
```

5.13.3. value

```
1 | def value(self) -> int
```

Return the grayscale threshold value (between 0 and 255).

item	description
type	func
return	returns grayscale threshold value
static	False

C++ definition code:

```
1 | int value()
```

5.13.4. l_value

```
1 | def l_value(self) -> int
```

Return the l channel threshold value of lab format (between 0 and 100).

item	description
type	func
return	returns l channel percentile value
static	False

C++ definition code:

```
1 | int l_value()
```

5.13.5. a_value

```
1 | def a_value(self) -> int
```

Return the a channel threshold value of lab format (between -128 and 127).

item	description
type	func
return	returns a channel percentile value
static	False

C++ definition code:

```
1 | int a_value()
```

5.13.6. b_value

```
1 | def b_value(self) -> int
```

Return the b channel threshold value of lab format (between -128 and 127).

item	description
type	func
return	returns b channel percentile value
static	False

C++ definition code:

```
1 | int b_value()
```

5.14. Histogram

Histogram class

C++ definition code:

```
1 | class Histogram
```

5.14.1. __init__

```
1 | def __init__(self, l_bin:  
    list[float], a_bin: list[float],  
    b_bin: list[float], format: Format =  
    ...) -> None
```

Histogram constructor

item	description
type	func
param	l_value : for grayscale image, it is grayscale threshold value list (the range of element values in the list is 0 and 255). for rgb888 image, it is l channel threshold value list of lab (the range of element values in the list is 0 and 100).

item	description
	a_value: for rgb888 image, it is a channel threshold value list of lab format(the range of element values in the list is -128 and 127). b_value: for rgb888 image, it is b channel threshold value list of lab format(the range of element values in the list is -128 and 127). format: format of the source image
static	False

C++ defination code:

```
1 | Histogram(std::vector<float>
   | l_bin, std::vector<float> a_bin,
   | std::vector<float> b_bin,
   | image::Format format =
   | image::Format::FMT_RGB888)
```

5.14.2. __getitem__

```
1 | def __getitem__(self, index: int) ->
   | int
```

Subscript operator

item	description
type	func
static	False

C++ defination code:

```
1 | int &__getitem__(int index)
```

5.14.3. bins

```
1 | def bins(self) -> list[float]
```

Returns a list of floats for the grayscale histogram.

item	description
type	func
static	False

C++ definition code:

```
1 | std::vector<float> bins()
```

5.14.4. l_bins

```
1 | def l_bins(self) -> list[float]
```

Returns a list of floats for the RGB565 histogram LAB L channel.

item	description
type	func
static	False

C++ definition code:

```
1 | std::vector<float> l_bins()
```

5.14.5. a_bins

```
1 | def a_bins(self) -> list[float]
```

Returns a list of floats for the RGB565 histogram LAB A channel.

item	description
type	func
static	False

C++ definition code:

```
1 | std::vector<float> a_bins()
```

5.14.6. b_bins

```
1 | def b_bins(self) -> list[float]
```

Returns a list of floats for the RGB565 histogram LAB B channel.

item	description
type	func
static	False

C++ definition code:

```
1 | std::vector<float> b_bins()
```

5.14.7. get_percentile

```
1 | def get_percentile(self, percentile:  
    float) -> Percentile
```

Computes the CDF of the histogram channels and returns a image::Percentile object

item	description
type	func
param	percentile: the values of the histogram at the passed in percentile (0.0 - 1.0) (float). So, if you pass in 0.1 this method will tell you (going from left-to-right in the histogram) what bin when summed into an accumulator caused the accumulator to cross 0.1. This is useful to determine min (with 0.1) and max (with 0.9) of a color distribution without outlier effects ruining your results for adaptive color tracking.
return	image::Percentile object
static	False

C++ defination code:

```
1 | image::Percentile  
   get_percentile(float percentile)
```

5.14.8. get_threshold

```
1 | def get_threshold(self) -> Threshold
```

Uses Otsu's Method to compute the optimal threshold values that split the histogram into two halves for each channel of the histogram and returns a image::Threshold object.

item	description
type	func
return	image::Threshold object
static	False

C++ definition code:

```
1 | image::Threshold get_threshold()
```

5.14.9. get_statistics

```
1 | def get_statistics(self) ->  
   Statistics
```

Computes the mean, median, mode, standard deviation, min, max, lower quartile, and upper quartile of each color channel in the histogram and returns a image::Statistics object.

item	description
type	func
return	image::Statistics object
static	False

C++ defination code:

```
1 | image::Statistics
   | get_statistics()
```

5.15. LBPKeyPoint

LBPKeyPoint class

C++ defination code:

```
1 | class LBPKeyPoint
```

5.15.1. __init__

```
1 | def __init__(self, data: list[int])
   | -> None
```

LBPKeyPoint constructor

item	description
type	func
param	data : The data of the LBPKeyPoint
static	False

C++ definition code:

```
1 | LBPKeyPoint(std::valarray<uint8_t>
    &data)
```

5.16. KeyPoint

KeyPoint class

C++ definition code:

```
1 | class KeyPoint
```

5.16.1. __init__

```
1 | def __init__(self, x: int, y: int,
    score: int, octave: int, angle: int,
    matched: int, desc: list[int]) ->
    None
```

KeyPoint constructor

item	description
type	func
param	x : The x of the KeyPoint y : The y of the KeyPoint score : The score of the KeyPoint octave : The octave of the KeyPoint angle : The angle of the KeyPoint matched : The matched of the

item	description
	KeyPoint desc: The desc of the KeyPoint
static	False

C++ defination code:

```
1 | KeyPoint(uint16_t x, uint16_t y,
   | uint16_t score, uint16_t octave,
   | uint16_t angle, uint16_t matched,
   | std::vector<uint8_t> &desc)
```

5.17. KPTMatch

KPTMatch class

C++ defination code:

```
1 | class KPTMatch
```

5.17.1. __init__

```
1 | def __init__(self, cx: int, cy: int,
   | x: int, y: int, w: int, h: int,
   | score: int, theta: int, match: int)
   | -> None
```

KPTMatch constructor

item	description
type	func

item	description
param	cx : The cx of the KPTMatch cy : The cy of the KPTMatch x : The x of the KPTMatch y : The y of the KPTMatch w : The w of the KPTMatch h : The h of the KPTMatch score : The score of the KPTMatch theta : The theta of the KPTMatch match : The match of the KPTMatch
static	False

C++ defination code:

```
1 | KPTMatch(int cx, int cy, int x,
    | int y, int w, int h, int score,
    | int theta, int match)
```

5.18. ORBKeyPoint

ORBKeyPoint class

C++ defination code:

```
1 | class ORBKeyPoint
```

5.18.1. __init__

```
1 | def __init__(self, data:
    | list[KeyPoint], threshold: int,
    | normalized: bool) -> None
```

ORBKeyPoint constructor

item	description
type	func
param	data : The data of the ORBKeyPoint threshold : The threshold of the ORBKeyPoint normalized : The normalized of the ORBKeyPoint
static	False

C++ defination code:

```
1 | ORBKeyPoint(std::vector<image::KeyPo  
   &data, int threshold, bool  
   normalized)
```

5.18.2. get_data

```
1 | def get_data(self) -> list[KeyPoint]
```

get data of ORBKeyPoint

item	description
type	func
return	return data of the ORBKeyPoint, type is std::vector
static	False

C++ defination code:

```
1 | std::vector<image::KeyPoint>  
   | get_data()
```

5.19. HaarCascade

HaarCascade class

C++ definition code:

```
1 | class HaarCascade
```

5.19.1. __init__

```
1 | def __init__(self) -> None
```

HaarCascade constructor

item	description
type	func
param	data : The data of the HaarCascade threshold : The threshold of the HaarCascade normalized : The normalized of the HaarCascade
static	False

C++ definition code:

```
1 | HaarCascade()
```

5.20. LineGroup

LineGroup class

C++ definition code:

```
1 | class LineGroup
```

5.20.1. __init__

```
1 | def __init__(self, id: int, type:  
    LineType, lines: list[Line]) -> None
```

LineGroup constructor

item	description
type	func
param	id : The id of line type : The line list type, @see image::LineType lines : The line list
static	False

C++ definition code:

```
1 | LineGroup(int id, image::LineType  
    type, std::vector<image::Line>  
    lines)
```

5.20.2. id

```
1 | def id(self) -> int
```

Get the line id of group, first id is 0.

item	description
type	func
return	return id
static	False

C++ defination code:

```
1 | int id()
```

5.20.3. type

```
1 | def type(self) -> LineType
```

Get the line type of group

item	description
type	func
return	returns line type. @see LineType
static	False

C++ defination code:

```
1 | image::LineType type()
```

5.20.4. lines

```
1 | def lines(self) -> list[Line]
```

Get a list of line

item	description
type	func
return	returns a list composed of Line objects
static	False

C++ defination code:

```
1 | std::vector<image::Line> lines()
```

5.21. Color

Color class

C++ defination code:

```
1 | class Color
```

5.21.1. __init__

```
1 | def __init__(self, ch1: int, ch2:  
    int = 0, ch3: int = 0, alpha: float  
    = 0, format: Format = ...) -> None
```

Color constructor

item	description
type	func
param	alpha : alpha channel, value range: 0 ~ 1
static	False

C++ defination code:

```
1 | Color(uint8_t ch1, uint8_t ch2 =  
   | 0, uint8_t ch3 = 0, float alpha =  
   | 0, image::Format format =  
   | image::FMT_GRAYSCALE)
```

5.21.2. r

Color red channel

item	description
type	var
static	False
readonly	False

C++ defination code:

```
1 | uint8_t r
```

5.21.3. g

Color green channel

item	description
type	var
static	False
readonly	False

C++ definition code:

```
1 | uint8_t g
```

5.21.4. b

Color blue channel

item	description
type	var
static	False
readonly	False

C++ definition code:

```
1 | uint8_t b
```

5.21.5. alpha

Color alpha channel, value from 0.0 to 1.0, float value

item	description
type	var

item	description
static	False
readonly	False

C++ defination code:

```
1 | float alpha
```

5.21.6. gray

Color gray channel

item	description
type	var
static	False
readonly	False

C++ defination code:

```
1 | uint8_t gray
```

5.21.7. format

Color format

item	description
type	var
static	False
readonly	False

C++ definition code:

```
1 | image::Format format
```

5.21.8. hex

```
1 | def hex(self) -> int
```

Get color's hex value

item	description
type	func
static	False

C++ definition code:

```
1 | uint32_t hex()
```

5.21.9. from_rgb

```
1 | def from_rgb(r: int, g: int, b: int)  
   -> Color
```

Create Color object from RGB channels

item	description
type	func
static	True

C++ definition code:

```
1 | static image::Color  
   | from_rgb(uint8_t r, uint8_t g,  
   | uint8_t b)
```

5.21.10. from_bgr

```
1 | def from_bgr(b: int, g: int, r: int)  
   | -> Color
```

Create Color object from BGR channels

item	description
type	func
static	True

C++ definition code:

```
1 | static image::Color  
   | from_bgr(uint8_t b, uint8_t g,  
   | uint8_t r)
```

5.21.11. from_gray

```
1 | def from_gray(gray: int) -> Color
```

Create Color object from gray channel

item	description
type	func
static	True

C++ definition code:

```
1 | static image::Color  
   from_gray(uint8_t gray)
```

5.21.12. from_rgba

```
1 | def from_rgba(r: int, g: int, b:  
   int, alpha: float) -> Color
```

Create Color object from RGBA channels

item	description
type	func
param	alpha : alpha channel, float value, value range: 0 ~ 1
static	True

C++ definition code:

```
1 | static image::Color  
   from_rgba(uint8_t r, uint8_t g,  
   uint8_t b, float alpha)
```

5.21.13. from_bgra

```
1 | def from_bgra(b: int, g: int, r:  
   int, alpha: float) -> Color
```

Create Color object from BGRA channels

item	description
type	func
param	alpha : alpha channel, float value, value range: 0 ~ 1
static	True

C++ definition code:

```
1 | static image::Color
   | from_bgra(uint8_t b, uint8_t g,
   | uint8_t r, float alpha)
```

5.21.14. from_hex

```
1 | def from_hex(hex: int, format:
   | Format) -> Color
```

Create Color object from hex value

item	description
type	func
param	hex : hex value, e.g. 0x0000FF00, lower address if first channel format : color format, @see image::Format
static	True

C++ definition code:

```
1 | static image::Color
   | from_hex(uint32_t hex,
```

```
image::Format &format)
```

5.21.15. to_format

```
1 | def to_format(self, format: Format)
   -> None
```

Convert Color format

item	description
type	func
param	format: format want to convert to, @see image::Format, only support RGB888, BGR888, RGBA8888, BGRA8888, GRAYSCALE.
static	False

C++ defination code:

```
1 | void to_format(const
   image::Format &format)
```

5.21.16. to_format2

```
1 | def to_format2(self, format: Format)
   -> Color
```

Convert color format and return a new Color object

item	description
type	func

item	description
param	format: format want to convert to, @see image::Format, only support RGB888, BGR888, RGBA8888, BGRA8888, GRAYSCALE.
return	new Color object, you need to delete it manually in C++.
static	False

C++ definition code:

```
1 | image::Color *to_format2(const
   | image::Format &format)
```

5.22. Image

Image class

C++ definition code:

```
1 | class Image
```

5.22.1. __init__

```
1 | def __init__(self, width: int,
   | height: int, format: Format = ...) -
   | > None
```

Image constructor

item	description
type	func
param	width : image width, should > 0 height : image height, should > 0 format : image format @see image::Format
static	False

C++ definition code:

```
1 | Image(int width, int height,
   | image::Format format =
   | image::Format::FMT_RGB888)
```

5.22.2. format

```
1 | def format(self) -> Format
```

Get image's format

item	description
type	func
see	image.Format
static	False

C++ definition code:

```
1 | image::Format format()
```

5.22.3. size

```
1 | def size(self) -> Size
```

Get image's size, [width, height]

item	description
type	func
static	False

C++ definition code:

```
1 | image::Size size()
```

5.22.4. data_size

```
1 | def data_size(self) -> int
```

Get image's data size

item	description
type	func
static	False

C++ definition code:

```
1 | int data_size()
```

5.22.5. width

```
1 | def width(self) -> int
```

Get image's width

item	description
type	func
static	False

C++ definition code:

```
1 | int width()
```

5.22.6. height

```
1 | def height(self) -> int
```

Get image's height

item	description
type	func
static	False

C++ definition code:

```
1 | int height()
```

5.22.7. __str__

```
1 | def __str__(self) -> str
```

To string method

item	description
type	func
static	False

C++ definition code:

```
1 | std::string __str__()
```

5.22.8. to_str

```
1 | def to_str(self) -> str
```

To string method

item	description
type	func
static	False

C++ definition code:

```
1 | std::string to_str()
```

5.22.9. get_pixel

```
1 | def get_pixel(self, x: int, y: int,
    rgbtuple: bool = False) -> list[int]
```

Get pixel of image

item	description
type	func
param	x: pixel's coordinate x. x must less than image's width y: pixel's coordinate y. y must less than image's height rgbtuple: switch return value method. rgbtuple decides whether to split the return or not. default is false.
return	pixel value, According to image format and rgbtuple, return different value: format is FMT_RGB888, rgbtuple is true, return [R, G, B]; rgbtuple is false, return [RGB] foramt is FMT_BGR888, rgbtuple is true, return [B, G, R]; rgbtuple is false, return [BGR] format is FMT_GRAYSCALE, return [GRAY];
static	False

C++ defination code:

```
1 | std::vector<int> get_pixel(int x,
   | int y, bool rgbtuple = false)
```

5.22.10. set_pixel

```
1 | def set_pixel(self, x: int, y: int,
   | pixel: list[int]) -> maix.err.Err
```

Set pixel of image

item	description
type	func
param	x: pixel's coordinate x. x must less than image's width y: pixel's coordinate y. y must less than image's height pixel: pixel value, according to image format and size of pixel, has different operation: format is FMT_RGB888, pixel size must be 1 or 3, if size is 1, will split pixel[0] to [R, G, B]; if size is 3, will use pixel directly format is FMT_BGR888, pixel size must be 1 or 3, if size is 1, will split pixel[0] to [B, G, R]; if size is 3, will use pixel directly format is FMT_GRAYSCALE, pixel size must be 1, will use pixel directly
return	error code, Err::ERR_NONE is ok, other is error
static	False

C++ defination code:

```
1 | err::Err set_pixel(int x, int y,  
   | std::vector<int> pixel)
```

5.22.11. to_tensor

```
1 | def to_tensor(self, chw: bool =  
    False, copy: bool = True) ->  
    maix.tensor.Tensor
```

Convert Image object to tensor::Tensor object

item	description
type	func
param	chw : if true, the shape of tensor is [C, H, W], else [H, W, C] copy : if true, will alloc memory for tensor data, else will use the memory of Image object
return	tensor::Tensor object pointer, an allocated tensor object
static	False

C++ defination code:

```
1 | tensor::Tensor *to_tensor(bool  
    chw = false, bool copy = true)
```

5.22.12. to_bytes

```
1 | def to_bytes(*args, **kwargs)
```

Get image's data and convert to array bytes

item	description
type	func
param	copy : if true, will alloc memory and copy data to new buffer, else will use the memory of Image object, delete bytes object will not affect Image object, but delete Image object will make bytes object invalid, it may cause program crash !!!! So use this param carefully.
return	image's data bytes, need be delete by caller in C++.
static	False

C++ defination code:

```
1 | Bytes *to_bytes(bool copy = true)
```

5.22.13. to_format

```
1 | def to_format(self, format: Format)
   -> Image
```

Convert image to specific format

item	description
type	func
param	format : format want to convert to, @see image::Format, only support

item	description
	RGB888, BGR888, RGBA8888, BGRA8888, GRAYSCALE, JPEG.
return	new image object. Need be delete by caller in C++.
throw	err.Exception, if two images' format not support, or already the format , will raise exception
static	False

C++ definition code:

```
1 | image::Image *to_format(const
   | image::Format &format)
```

5.22.14. to_jpeg

```
1 | def to_jpeg(self, quality: int = 95)
   | -> Image
```

Convert image to jpeg

item	description
type	func
param	quality : the quality of jpg, default is 95. For MaixCAM supported range is (50, 100], if ≤ 50 will be fixed to 51.
return	new image object. Need be delete by caller in C++.

item	description
throw	err.Exception, if two images' format not support, or already the format , will raise exception
static	False

C++ defination code:

```
1 | image::Image *to_jpeg(int quality
   | = 95)
```

5.22.15. draw_image

```
1 | def draw_image(self, x: int, y: int,
   | img: Image) -> Image
```

Draw image on this image

item	description
type	func
param	x: left top corner of image point's coordinate x y: left top corner of image point's coordinate y img: image object to draw, the caller's channel must $\leq$ the args' channel, e.g. caller is RGB888, args is RGBA8888, will throw exception, but caller is RGBA8888, args is RGB888 or RGBA8888 is ok

item	description
return	this image object self
static	False
C++ defination code:	
<pre> 1 image::Image *draw_image(int x, int y, image::Image &img) </pre>	

5.22.16. draw_rect

```

1 | def draw_rect(self, x: int, y: int,
   |   w: int, h: int, color: Color,
   |   thickness: int = 1) -> Image

```

Fill rectangle color to image

item	description
type	func
param	x: left top corner of rectangle point's coordinate x y: left top corner of rectangle point's coordinate y w: rectangle width h: rectangle height color: rectangle color thickness: rectangle thickness(line width), by default(value is 1), -1 means fill rectangle
return	this image object self

item	description
static	False
C++ defination code:	
<pre> 1 image::Image *draw_rect(int x, int y, int w, int h, const image::Color &color, int thickness = 1) </pre>	

5.22.17. draw_line

```

1 | def draw_line(self, x1: int, y1:
    |   int, x2: int, y2: int, color: Color,
    |   thickness: int = 1) -> Image

```

Draw line on image

item	description
type	func
param	x1 : start point's coordinate x y1 : start point's coordinate y x2 : end point's coordinate x y2 : end point's coordinate y color : line color @see image::Color thickness : line thickness(line width), by default(value is 1)
return	this image object self
static	False
C++ defination code:	

```
1 | image::Image *draw_line(int x1,
    int y1, int x2, int y2, const
    image::Color &color, int
    thickness = 1)
```

5.22.18. draw_circle

```
1 | def draw_circle(self, x: int, y:
    int, radius: int, color: Color,
    thickness: int = 1) -> Image
```

Draw circle on image

item	description
type	func
param	x : circle center point's coordinate x y : circle center point's coordinate y radius : circle radius color : circle color @see image::Color thickness : circle thickness(line width), default -1 means fill circle
return	this image object self
static	False

C++ defination code:

```
1 | image::Image *draw_circle(int x,
    int y, int radius, const
    image::Color &color, int
    thickness = 1)
```

5.22.19. draw_ellipse

```

1 | def draw_ellipse(self, x: int, y:
    int, a: int, b: int, angle: float,
    start_angle: float, end_angle:
    float, color: Color, thickness: int
    = 1) -> Image

```

Draw ellipse on image

item	description
type	func
param	x : ellipse center point's coordinate x y : ellipse center point's coordinate y a : ellipse major axis length b : ellipse minor axis length angle : ellipse rotation angle start_angle : ellipse start angle end_angle : ellipse end angle color : ellipse color @see image::Color thickness : ellipse thickness(line width), by default(value is 1), -1 means fill ellipse
return	this image object self
static	False

C++ defination code:

```

1 | image::Image *draw_ellipse(int x,
    int y, int a, int b, float angle,
    float start_angle, float
    end_angle, const image::Color
    &color, int thickness = 1)

```

5.22.20. draw_string

```
1 | def draw_string(self, x: int, y:  
  | int, textstring: str, color: Color =  
  | ..., scale: float = 1, thickness:  
  | int = -1, wrap: bool = True,  
  | wrap_space: int = 4, font: str = '')  
  | -> Image
```

Draw text on image

item	description
type	func
param	x: text left top point's coordinate x y: text left top point's coordinate y string: text content color: text color @see image::Color, default is white scale: font scale, by default(value is 1) thickness: text thickness(line width), if negative, the glyph is filled, by default(value is -1) wrap: if true, will auto wrap text to next line if text width > image width, by default(value is true)
return	this image object self
static	False

C++ defination code:

```
1 | image::Image *draw_string(int x,  
2 | int y, const std::string  
  | &textstring, const image::Color
```

```

&color = image::COLOR_WHITE,
float scale = 1, int thickness =
-1,

bool wrap = true, int wrap_space
= 4, const std::string &font =
"")

```

5.22.21. draw_cross

```

1 | def draw_cross(self, x: int, y: int,
   | color: Color, size: int = 5,
   | thickness: int = 1) -> Image

```

Draw cross on image

item	description
type	func
param	x: cross center point's coordinate x y: cross center point's coordinate y color: cross color @see image::Color size: how long the lines of the cross extend, by default(value is 5). So the line length is <code>2 * size + thickness</code> thickness: cross thickness(line width), by default(value is 1)
static	False

C++ defination code:

```

1 | image::Image *draw_cross(int x,
   | int y, const image::Color &color,
   | int size = 5, int thickness = 1)

```

5.22.22. draw_arrow

```
1 | def draw_arrow(self, x0: int, y0:
    int, x1: int, y1: int, color: Color,
    thickness: int = 1) -> Image
```

Draw arrow on image

item	description
type	func
param	x0 : start coordinate of the arrow x0 y0 : start coordinate of the arrow y0 x1 : end coordinate of the arrow x1 y1 : end coordinate of the arrow y1 color : cross color @see image::Color thickness : cross thickness(line width), by default(value is 1)
return	this image object self
static	False

C++ defination code:

```
1 | image::Image *draw_arrow(int x0,
    int y0, int x1, int y1, const
    image::Color &color, int
    thickness = 1)
```

5.22.23. draw_edges

```
1 | def draw_edges(self, corners:
    list[list[int]], color: Color, size:
    int = 0, thickness: int = 1, fill:
```

```
bool = False) -> Image
```

Draw edges on image

item	description
type	func
param	corners: edges, [[x0, y0], [x1, y1], [x2, y2], [x3, y3]] color: edges color @see image::Color size: the circle of radius size. TODO: support in the feature thickness: edges thickness(line width), by default(value is 1) fill: if true, will fill edges, by default(value is false)
return	this image object self
static	False

C++ definition code:

```
1 | image::Image  
   | *draw_edges(std::vector<std::vector<  
   | corners, const image::Color  
   | &color, int size = 0, int  
   | thickness = 1, bool fill = false)
```

5.22.24. draw_keypoints

```
1 | def draw_keypoints(self, keypoints:  
   | list[int], color: Color, size: int =  
   | 4, thickness: int = -1,  
   | line_thickness: int = 0) -> Image
```

Draw keypoints on image

item	description
type	func
param	keypoints: keypoints, [x1, y1, x2, y2...] or [x, y, rotation_andle_in_degrees, x2, y2, rotation_andle_in_degrees2] (TODO: rotation_andle_in_degrees support in the feature) color: keypoints color @see image::Color size: size of keypoints(radius) thickness: keypoints thickness(line width), by default(value is -1 means fill circle) line_thickness: line thickness, default 0 means not draw lines, > 0 will draw lines connect points.
return	this image object self
static	False

C++ defination code:

```
1 | image::Image
   | *draw_keypoints(const
   | std::vector<int> &keypoints,
   | const image::Color &color, int
   | size = 4, int thickness = -1, int
   | line_thickness = 0)
```

5.22.25. resize

```

1 | def resize(self, width: int, height:
    | int, object_fit: Fit = ..., method:
    | ResizeMethod = ...) -> Image

```

Resize image, will create a new resized image object

item	description		
type	func		
param	width: new width, if value is -1, will use height to calculate aspect ratio height: new height, if value is -1, will use width to calculate aspect ratio object_fit: fill, contain, cover, by default is fill method: resize method, by default is NEAREST		
return	Always return a new resized image object		 -> height() != height:  =  -

item	description	
	even size not change, So in C++ you should take care of the return value to avoid memory leak. And it's better to judge whether the size has changed before calling this function to make the program more efficient. e.g. if <code>img->width() != width</code>	<pre>>resize(width, height);</pre>
static	False	

C++ defination code:

```
1 | image::Image *resize(int width,
   | int height, image::Fit object_fit
   | = image::Fit::FIT_FILL,
   | image::ResizeMethod method =
   | image::ResizeMethod::NEAREST)
```

5.22.26. affine

```

1 | def affine(self, src_points:
    | list[int], dst_points: list[int],
    | width: int = -1, height: int = -1,
    | method: ResizeMethod = ...) -> Image

```

Affine transform image, will create a new transformed image object, need 3 points.

item	description
type	func
param	src_points: three source points, [x1, y1, x2, y2, x3, y3] dst_points: three destination points, [x1, y1, x2, y2, x3, y3] width: new width, if value is -1, will use height to calculate aspect ratio height: new height, if value is -1, will use width to calculate aspect ratio method: resize method, by default is bilinear
return	new transformed image object
static	False

C++ definition code:

```

1 | image::Image
    | *affine(std::vector<int>
    | src_points, std::vector<int>
    | dst_points, int width = -1, int
    | height = -1, image::ResizeMethod
    | method =
    | image::ResizeMethod::BILINEAR)

```

5.22.27. perspective

```
1 | def perspective(self, src_points:  
    list[int], dst_points: list[int],  
    width: int = -1, height: int = -1,  
    method: ResizeMethod = ...) -> Image
```

Perspective transform image, will create a new transformed image object, need 4 points.

item	description
type	func
param	src_points : three source points, [x1, y1, x2, y2, x3, y3, x4, y4] dst_points : three destination points, [x1, y1, x2, y2, x3, y3, x4, y4] width : new width, if value is -1, will use height to calculate aspect ratio height : new height, if value is -1, will use width to calculate aspect ratio method : resize method, by default is bilinear
return	new transformed image object
static	False

C++ definition code:

```
1 | image::Image*  
    perspective(std::vector<int>  
    src_points, std::vector<int>  
    dst_points, int width = -1, int  
    height = -1, image::ResizeMethod  
    method =
```

```
image::ResizeMethod::BILINEAR)
```

5.22.28. copy

```
1 | def copy(self) -> Image
```

Copy image, will create a new copied image object

item	description
type	func
return	new copied image object
static	False

C++ defination code:

```
1 | image::Image *copy()
```

5.22.29. crop

```
1 | def crop(self, x: int, y: int, w: int, h: int) -> Image
```

Crop image, will create a new cropped image object

item	description
type	func
param	x: left top corner of crop rectangle point's coordinate x y: left top corner of crop rectangle

item	description
	point's coordinate y w: crop rectangle width h: crop rectangle height
return	new cropped image object
static	False

C++ defination code:

```
1 | image::Image *crop(int x, int y,
   | int w, int h)
```

5.22.30. rotate

```
1 | def rotate(self, angle: float,
   | width: int = -1, height: int = -1,
   | method: ResizeMethod = ...) -> Image
```

Rotate image, will create a new rotated image object

item	description
type	func
param	angle: anti-clock wise rotate angle, if angle is 90 or 270, and width or height is -1, will swap width and height, or will throw exception width: new width, if value is -1, will use height to calculate aspect ratio height: new height, if value is -1, will use width to calculate aspect ratio

item	description
	method: resize method, by default is bilinear
return	new rotated image object
static	False

C++ definition code:

```
1 | image::Image *rotate(float angle,
  |   int width = -1, int height = -1,
  |   image::ResizeMethod method =
  |   image::ResizeMethod::BILINEAR)
```

5.22.31. flip

```
1 | def flip(self, dir: FlipDir) ->
  |   Image
```

Vertical flip image, and return a new image.

item	description
type	func
param	dir: flip dir, see image.FlipDir, e.g. image.FlipDir.X is vertical flip.
return	new flipped image.
throw	When arg error, will throw out err.Err exception.
static	False

C++ definition code:

```
1 | image::Image *flip(const
    image::FlipDir dir)
```

5.22.32. mean_pool

```
1 | def mean_pool(self, x_div: int,
    y_div: int, copy: bool = False) ->
    Image
```

Finds the mean of $x_div * y_div$ squares in the image and returns the modified image composed of the mean of each square.

item	description
type	func
param	x_div: The width of the squares. y_div: The height of the squares. copy: Select whether to return a new image or modify the original image. default is false. If true, returns a new image composed of the mean of each square; If false, returns the modified image composed of the mean of each square.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```
1 | image::Image *mean_pool(int
    x_div, int y_div, bool copy =
    false)
```

5.22.33. midpoint_pool

```
1 | def midpoint_pool(self, x_div: int,
    y_div: int, bias: float = 0.5, copy:
    bool = False) -> Image
```

Finds the midpoint of $x_div * y_div$ squares in the image and returns the modified image composed of the mean of each square.

item	description
type	func
param	x_div: The width of the squares. y_div: The height of the squares. bias: The bias of the midpoint. default is 0.5. midpoint value is equal to $(max * bias + min * (1 - bias))$ copy: Select whether to return a new image or modify the original image. default is false. If true, returns a new image composed of the midpoint of each square; If false, returns the modified image composed of the midpoint of each square.

item	description
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *midpoint_pool(int
   | x_div, int y_div, double bias =
   | 0.5, bool copy = false)
```

5.22.34. compress

```
1 | def compress(self, quality: int =
   | 95) -> Image
```

JPEG compresses the image in place, the same as to_jpeg function, it's recommend to use to_jpeg instead.

item	description
type	func
param	quality: The quality of the compressed image. default is 95.
return	Returns the compressed JPEG image
static	False

C++ definition code:

```
1 | image::Image *compress(int
   | quality = 95)
```

5.22.35. clear

```
1 | def clear(self, mask: Image = None)
   -> Image
```

Sets all pixels in the image to zero

item	description
type	func
param	mask : Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```
1 | image::Image *clear(image::Image
   *mask = nullptr)
```

5.22.36. mask_rectangle

```
1 | def mask_rectangle(self, x: int = -1,
   y: int = -1, w: int = -1, h: int =
```

```
-1) -> Image
```

Zeros a rectangular part of the image. If no arguments are supplied this method zeros the center of the image.

item	description
type	func
param	x : The x coordinate of the top left corner of the rectangle. y : The y coordinate of the top left corner of the rectangle. w : The width of the rectangle. h : The height of the rectangle.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *mask_rectangle(int x  
  | = -1, int y = -1, int w = -1, int  
  | h = -1)
```

5.22.37. mask_circle

```
1 | def mask_circle(self, x: int = -1,  
  | y: int = -1, radius: int = -1) ->  
  | Image
```

Zeros a circular part of the image. If no arguments are supplied this method zeros the center of the

image.

item	description
type	func
param	x : The x coordinate of the center of the circle. y : The y coordinate of the center of the circle. radius : The radius of the circle.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *mask_circle(int x =  
  -1, int y = -1, int radius = -1)
```

5.22.38. mask_ellipse

```
1 | def mask_ellipse(self, x: int = -1,  
  y: int = -1, radius_x: int = -1,  
  radius_y: int = -1,  
  rotation_angle_in_degrees: float =  
  0) -> Image
```

Zeros a ellipse part of the image. If no arguments are supplied this method zeros the center of the image.

item	description
type	func
param	x: The x coordinate of the center of the ellipse. y: The y coordinate of the center of the ellipse. radius_x: The radius of the ellipse in the x direction. radius_y: The radius of the ellipse in the y direction. rotation_angle_in_degrees: The rotation angle of the ellipse in degrees.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *mask_ellipse(int x
   | = -1, int y = -1, int radius_x =
   | -1, int radius_y = -1, float
   | rotation_angle_in_degrees = 0)
```

5.22.39. binary

```
1 | def binary(self, thresholds:
   | list[list[int]] = [], invert: bool =
   | False, zero: bool = False, mask:
   | Image = None, to_bitmap: bool =
   | False, copy: bool = False) -> Image
```

Sets all pixels in the image to black or white depending on if the pixel is inside of a threshold in the threshold list thresholds or not.

item	description
type	func
note	For GRAYSCALE format, Lmin and Lmax range is [0, 255]. For RGB888 format, Lmin and Lmax range is [0, 100].
param	thresholds: You can define multiple thresholds. For GRAYSCALE format, you can use {{Lmin, Lmax}, ...} to define one or more thresholds. For RGB888 format, you can use {{Lmin, Lmax, Amin, Amax, Bmin, Bmax}, ...} to define one or more thresholds. Where the upper case L,A,B represent the L,A,B channels of the LAB image format, and min, max represent the minimum and maximum values of the corresponding channels. invert: If true, the thresholds will be inverted before the operation. default is false. zero: If zero is true, the image will be set the pixels within the threshold to 0, other pixels remain unchanged. If zero is false, the image will be set to



item	description
	black or white. default is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None. to_bitmap: If true, the image will be converted to a bitmap image before thresholding. default is false. TODO: support in the feature copy: Select whether to return a new image or modify the original image. default is false.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```

1 | image::Image
   | *binary(std::vector<std::vector<int>>(),
   | thresholds =
   | std::vector<std::vector<int>>(),
   | bool invert = false, bool zero =
   | false, image::Image *mask =
   | nullptr, bool to_bitmap = false,
   | bool copy = false)

```

5.22.40. invert

```
1 | def invert(self) -> Image
```

Inverts the image in place.

item	description
type	func
return	Returns the image after the operation is completed
static	False

C++ defination code:

```
1 | image::Image *invert()
```

5.22.41. b_and

```
1 | def b_and(self, other: Image, mask:  
    Image = None) -> Image
```

Performs a bitwise and operation between the image and the other image.

item	description
type	func
param	other: The other image should be an image and should be the same size as the image being operated on. TODO: support path?

item	description
	mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```
1 | image::Image *b_and(image::Image
   | *other, image::Image *mask =
   | nullptr)
```

5.22.42. b_nand

```
1 | def b_nand(self, other: Image, mask:
   | Image = None) -> Image
```

Performs a bitwise nand operation between the image and the other image.

item	description
type	func
param	other: The other image should be an image and should be the same size as

item	description
	the image being operated on. TODO: support path? mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *b_nand(image::Image
    | *other, image::Image *mask =
    | nullptr)
```

5.22.43. b_or

```
1 | def b_or(self, other: Image, mask:
    | Image = None) -> Image
```

Performs a bitwise or operation between the image and the other image.

item	description
type	func

item	description
param	other: The other image should be an image and should be the same size as the image being operated on. TODO: support path? mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *b_or(image::Image
   | *other, image::Image *mask =
   | nullptr)
```

5.22.44. b_nor

```
1 | def b_nor(self, other: Image, mask:
   | Image = None) -> Image
```

Performs a bitwise nor operation between the image and the other image.

item	description
type	func
param	other: The other image should be an image and should be the same size as the image being operated on. TODO: support path? mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *b_nor(image::Image
   | *other, image::Image *mask =
   | nullptr)
```

5.22.45. b_xor

```
1 | def b_xor(self, other: Image, mask:
   | Image = None) -> Image
```

Performs a bitwise xor operation between the image and the other image.

item	description
type	func
param	other: The other image should be an image and should be the same size as the image being operated on. TODO: support path? mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *b_xor(image::Image
   | *other, image::Image *mask =
   | nullptr)
```

5.22.46. b_xnor

```
1 | def b_xnor(self, other: Image, mask:
   | Image = None) -> Image
```

Performs a bitwise xnor operation between the image and the other image.

item	description
type	func
param	other: The other image should be an image and should be the same size as the image being operated on. TODO: support path? mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *b_xnor(image::Image
   | *other, image::Image *mask =
   | nullptr)
```

5.22.47. awb

```
1 | def awb(self, max: bool = False) ->
   | Image
```

Performs an auto white balance operation on the image. TODO: support in the feature

item	description
type	func
param	max: if True uses the white-patch algorithm instead. default is false.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *awb(bool max =
   | false)
```

5.22.48. ccm

```
1 | def ccm(self, matrix: list[float]) -
   | > Image
```

Multiplies the passed (3x3) or (4x3) floating-point color-correction-matrix with the image.\nnote: Grayscale format is not support.

item	description
type	func
param	matrix: The color correction matrix to use. 3x3 or 4x3 matrix. Weights may either be positive or negative, and the sum of each column in the 3x3 matrix should generally be 1.

item	description
	example: <pre>{ 1, 0, 0, 0, 1, 0, 0, 0, 1, }</pre> Where the last row of the 4x3 matrix is an offset per color channel. If you add an offset you may wish to make the weights sum to less than 1 to account for the offset. example: <pre>{ 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, }</pre>
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image
   | *ccm(std::vector<float> &matrix)
```

5.22.49. gamma

```
1 | def gamma(self, gamma: float = 1.0,
    contrast: float = 1.0, brightness:
    float = 0.0) -> Image
```

Quickly changes the image gamma, contrast, and brightness. Create a array whose size is usually 255,\nand use the parameters gamma, contrast, and brightness to calculate the value of the array, and then map the\nimage pixel value through the value of the array.\nThe calculation method for array is: $\text{array}[\text{array_idx}] = (\text{powf}((\text{array_idx} / 255.0), (1 / \text{gamma})) * \text{contrast} + \text{brightness}) * \text{scale}$,\n `powf` is a function used to calculate floating point power.\n `array` is the array used for mapping.\n `array_idx` is the index of the array, the maximum value is determined according to the image format, usually 255.\n `scale` is a constant, the value is determined by the image format, usually 255.\nMapping method:\nAssume that a pixel value in the image is 128, then map the pixel value to the value of `array[128]`\nUsers can adjust the value of the array through the gamma, contrast, and brightness parameters.

item	description
type	func
param	gamma: The contrast gamma greater than 1.0 makes the image darker in a non-linear manner while less than 1.0 makes the image brighter. default is 1.0. contrast: The contrast value greater

item	description
	than 1.0 makes the image brighter in a linear manner while less than 1.0 makes the image darker. default is 1.0. brightness: The brightness value greater than 0.0 makes the image brighter in a constant manner while less than 0.0 makes the image darker. default is 0.0.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *gamma(double gamma
    = 1.0, double contrast = 1.0,
    double brightness = 0.0)
```

5.22.50. gamma_corr

```
1 | def gamma_corr(self, gamma: float,
    contrast: float = 1.0, brightness:
    float = 0.0) -> Image
```

Alias for Image.gamma.

item	description
type	func

item	description
param	gamma: The contrast gamma greater than 1.0 makes the image darker in a non-linear manner while less than 1.0 makes the image brighter. default is 1.0. contrast: The contrast value greater than 1.0 makes the image brighter in a linear manner while less than 1.0 makes the image darker. default is 1.0. brightness: The brightness value greater than 0.0 makes the image brighter in a constant manner while less than 0.0 makes the image darker. default is 0.0.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *gamma_corr(double
    gamma, double contrast = 1.0,
    double brightness = 0.0)
```

5.22.51. negate

```
1 | def negate(self) -> Image
```

Flips (numerically inverts) all pixels values in an image

item	description
type	func
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *negate()
```

5.22.52. replace

```
1 | def replace(self, other: Image =  
    None, hmirror: bool = False, vflip:  
    bool = False, transpose: bool =  
    False, mask: Image = None) -> Image
```

Replaces all pixels in the image with the corresponding pixels in the other image.

item	description
type	func
param	other: The other image should be an image and should be the same size as the image being operated on. hmirror: If true, the image will be horizontally mirrored before the operation. default is false.

item	description
	vflip: If true, the image will be vertically flipped before the operation. default is false. transpose: If true, the image can be used to rotate 90 degrees or 270 degrees. hmirror = false, vflip = false, transpose = false, the image will not be rotated. hmirror = false, vflip = true, transpose = true, the image will be rotated 90 degrees. hmirror = true, vflip = true, transpose = false, the image will be rotated 180 degrees. hmirror = true, vflip = false, transpose = true, the image will be rotated 270 degrees. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False



C++ definition code:

```
1 | image::Image  
   | *replace(image::Image *other =  
   | nullptr, bool hmirror = false,  
   | bool vflip = false, bool  
   | transpose = false, image::Image  
   | *mask = nullptr)
```

5.22.53. set

```
1 | def set(self, other: Image, hmirror:  
   | bool = False, vflip: bool = False,  
   | transpose: bool = False, mask: Image  
   | = None) -> Image
```

Alias for Image::replace.

item	description
type	func
param	other: The other image should be an image and should be the same size as the image being operated on. hmirror: If true, the image will be horizontally mirrored before the operation. default is false. vflip: If true, the image will be vertically flipped before the operation. default is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels

item	description
	and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *set(image::Image
    | *other, bool hmirror = false,
    | bool vflip = false, bool
    | transpose = false, image::Image
    | *mask = nullptr)
```

5.22.54. add

```
1 | def add(self, other: Image, mask:
    | Image = None) -> Image
```

Adds the other image to the image.

item	description
type	func
param	other: The other image should be an image and should be the same size as the image being operated on. TODO: support path? mask: Mask is another image to use

item	description
	as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *add(image::Image
   | *other, image::Image *mask =
   | nullptr)
```

5.22.55. sub

```
1 | def sub(self, other: Image, reverse:
   | bool = False, mask: Image = None) ->
   | Image
```

Subtracts the other image from the image.

item	description
type	func
param	other: The other image should be an image and should be the same size as the image being operated on. TODO:

item	description
	support path? reverse: If true, the image will be reversed before the operation. default is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *sub(image::Image
    | *other, bool reverse = false,
    | image::Image *mask = nullptr)
```

5.22.56. mul

```
1 | def mul(self, other: Image, invert:
    | bool = False, mask: Image = None) ->
    | Image
```

Multiplies the image by the other image.\nNote:
This method is meant for image blending and

cannot multiply the pixels in the image by a scalar like 2.

item	description
type	func
param	other: The other image should be an image and should be the same size as the image being operated on. TODO: support path? invert: If true, the image will be change the multiplication operation from ab to $1/((1/a)(1/b))$. In particular, this lightens the image instead of darkening it (e.g. multiply versus burn operations). default is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```
1 | image::Image *mul(image::Image
    *other, bool invert = false,
    image::Image *mask = nullptr)
```

5.22.57. div

```
1 | def div(self, other: Image, invert:
    bool = False, mod: bool = False,
    mask: Image = None) -> Image
```

Divides the image by the other image.\nThis method is meant for image blending and cannot divide the pixels in the image by a scalar like 2.

item	description
type	func
param	other: The other image should be an image and should be the same size as the image being operated on. TODO: support path? invert: If true, the image will be change the division direction from a/b to b/a. default is false. mod: If true, the image will be change the division operation to the modulus operation. default is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on.

item	description
	Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```
1 | image::Image *div(image::Image
   | *other, bool invert = false, bool
   | mod = false, image::Image *mask =
   | nullptr)
```

5.22.58. min

```
1 | def min(self, other: Image, mask:
   | Image = None) -> Image
```

Calculate the minimum of each pixel in the image and the other image.

item	description
type	func
param	other: The other image should be an image and should be the same size as the image being operated on. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels

item	description
	and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *min(image::Image
   | *other, image::Image *mask =
   | nullptr)
```

5.22.59. max

```
1 | def max(self, other: Image, mask:
   | Image = None) -> Image
```

Calculate the maximum of each pixel in the image and the other image.

item	description
type	func
param	other: The other image should be an image and should be the same size as the image being operated on. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an

item	description
	image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *max(image::Image
   | *other, image::Image *mask =
   | nullptr)
```

5.22.60. difference

```
1 | def difference(self, other: Image,
   | mask: Image = None) -> Image
```

Calculate the absolute value of the difference between each pixel in the image and the other image.

item	description
type	func
param	other: The other image should be an image and should be the same size as the image being operated on. mask: Mask is another image to use

item	description
	as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image
   | *difference(image::Image *other,
   | image::Image *mask = nullptr)
```

5.22.61. blend

```
1 | def blend(self, other: Image, alpha:
   | int = 128, mask: Image = None) ->
   | Image
```

Blends the image with the other image.\nres =
alpha * this_img / 256 + (256 - alpha) * other_img /
256

item	description
type	func

item	description
param	other: The other image should be an image and should be the same size as the image being operated on. alpha: The alpha value of the blend, the value range is [0, 256], default is 128. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *blend(image::Image
   | *other, int alpha = 128,
   | image::Image *mask = nullptr)
```

5.22.62. histeq

```
1 | def histeq(self, adaptive: bool =
   | False, clip_limit: int = -1, mask:
   | Image = None) -> Image
```

Runs the histogram equalization algorithm on the image.

item	description
type	func
param	adaptive: If true, an adaptive histogram equalization method will be run on the image instead which as generally better results than non-adaptive histogram equalization but a longer run time. default is false. clip_limit: Provides a way to limit the contrast of the adaptive histogram equalization. Use a small value for this, like 10, to produce good histogram equalized contrast limited images. default is -1. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *histeq(bool
    adaptive = false, int clip_limit
    = -1, image::Image *mask =
    nullptr)
```

5.22.63. mean

```
1 | def mean(self, size: int, threshold:
    bool = False, offset: int = 0,
    invert: bool = False, mask: Image =
    None) -> Image
```

Standard mean blurring filter using a box filter.\n\nThe parameters offset and invert are valid when threshold is True.

item	description
type	func
param	size: Kernel size. The actual kernel size is $((size * 2) + 1) * ((size * 2) + 1)$. Use 1(3x3 kernel), 2(5x5 kernel). threshold: If true, which will enable adaptive thresholding of the image which sets pixels to white or black based on a pixel' s brightness in relation to the brightness of the kernel of pixels around them. default is false. offset: The larger the offset value, the lower brightness pixels on the original image will be set to white. default is 0.

item	description
	invert: If true, the image will be inverted before the operation. default is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```
1 | image::Image *mean(int size, bool
   | threshold = false, int offset =
   | 0, bool invert = false,
   | image::Image *mask = nullptr)
```

5.22.64. median

```
1 | def median(self, size: int,
   | percentile: float = 0.5, threshold:
   | bool = False, offset: int = 0,
   | invert: bool = False, mask: Image =
   | None) -> Image
```

Runs the median filter on the image. The median filter is the best filter for smoothing surfaces while preserving edges but it is very slow.

item	description
type	func
param	size: Kernel size. The actual kernel size is $((size * 2) + 1) * ((size * 2) + 1)$. Use 1(3x3 kernel), 2(5x5 kernel). percentile: This parameter controls the percentile of the value used in the kernel. You can set this to 0 for a min filter, 0.25 for a lower quartile filter, 0.75 for an upper quartile filter, and 1.0 for a max filter. default is 0.5. threshold: If true, which will enable adaptive thresholding of the image which sets pixels to white or black based on a pixel' s brightness in relation to the brightness of the kernel of pixels around them. default is false. offset: The larger the offset value, the lower brightness pixels on the original image will be set to white. default is 0. invert: If true, the image will be inverted before the operation. default is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an



item	description
	image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *median(int size,
    | double percentile = 0.5, bool
    | threshold = false, int offset =
    | 0, bool invert = false,
    | image::Image *mask = nullptr)
```

5.22.65. mode

```
1 | def mode(self, size: int, threshold:
    | bool = False, offset: int = 0,
    | invert: bool = False, mask: Image =
    | None) -> Image
```

Runs the mode filter on the image by replacing each pixel with the mode of their neighbors.

item	description
type	func
param	size: Kernel size. The actual kernel size is $((size * 2) + 1) * ((size * 2) + 1)$.

item	description
	Use 1(3x3 kernel), 2(5x5 kernel). threshold: If true, which will enable adaptive thresholding of the image which sets pixels to white or black based on a pixel' s brightness in relation to the brightness of the kernel of pixels around them. default is false. offset: The larger the offset value, the lower brightness pixels on the original image will be set to white. default is 0. invert: If true, the image will be inverted before the operation. default is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```
1 | image::Image *mode(int size, bool
   | threshold = false, int offset =
```

```
0, bool invert = false,
image::Image *mask = nullptr)
```

5.22.66. midpoint

```
1 | def midpoint(self, size: int, bias:
    float = 0.5, threshold: bool =
    False, offset: int = 0, invert: bool
    = False, mask: Image = None) ->
    Image
```

Runs the midpoint filter on the image. This filter finds the midpoint ($\text{max} * \text{bias} + \text{min} * (1 - \text{bias})$) of each pixel neighborhood in the image.

item	description
type	func
param	size: Kernel size. The actual kernel size is $((\text{size} * 2) + 1) * ((\text{size} * 2) + 1)$. Use 1(3x3 kernel), 2(5x5 kernel). bias: The bias of the midpoint. default is 0.5. threshold: If true, which will enable adaptive thresholding of the image which sets pixels to white or black based on a pixel' s brightness in relation to the brightness of the kernel of pixels around them. default is false. offset: The larger the offset value, the lower brightness pixels on the original image will be set to white. default is 0.

item	description
	invert: If true, the image will be inverted before the operation. default is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *midpoint(int size,
    | double bias = 0.5, bool threshold
    | = false, int offset = 0, bool
    | invert = false, image::Image
    | *mask = nullptr)
```

5.22.67. morph

```
1 | def morph(self, size: int, kernel:
    | list[int], mul: float = -1, add:
    | float = 0.0, threshold: bool =
    | False, offset: int = 0, invert: bool
    | = False, mask: Image = None) ->
    | Image
```

Convolves the image by a filter kernel. This allows you to do general purpose convolutions on an image.

item	description
type	func
param	size: Kernel size. The actual kernel size is $((size * 2) + 1) * ((size * 2) + 1)$. Use 1(3x3 kernel), 2(5x5 kernel). kernel: The kernel used for convolution. The kernel should be a list of lists of numbers. The kernel should be the same size as the actual kernel size. mul: This parameter is used to multiply the convolved pixel results. default is auto. add: This parameter is the value to be added to each convolution pixel result. default is 0.0. threshold: If true, which will enable adaptive thresholding of the image which sets pixels to white or black based on a pixel' s brightness in relation to the brightness of the kernel of pixels around them. default is false. offset: The larger the offset value, the lower brightness pixels on the original image will be set to white. default is 0. invert: If true, the image will be

item	description
	inverted before the operation. default is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *morph(int size,
    | std::vector<int> kernel, float
    | mul = -1, float add = 0.0, bool
    | threshold = false, int offset =
    | 0, bool invert = false,
    | image::Image *mask = nullptr)
```

5.22.68. gaussian

```
1 | def gaussian(self, size: int,
    | unsharp: bool = False, mul: float =
    | -1, add: float = 0.0, threshold:
    | bool = False, offset: int = 0,
    | invert: bool = False, mask: Image =
    | None) -> Image
```

Convolves the image by a smoothing gaussian kernel.

item	description
type	func
param	size: Kernel size. The actual kernel size is $((size * 2) + 1) * ((size * 2) + 1)$. Use 1(3x3 kernel), 2(5x5 kernel). unsharp: If true, this method will perform an unsharp mask operation instead of gaussian filtering operation, this improves the clarity of image edges. default is false. mul: This parameter is used to multiply the convolved pixel results. default is auto. add: This parameter is the value to be added to each convolution pixel result. default is 0.0. threshold: If true, which will enable adaptive thresholding of the image which sets pixels to white or black based on a pixel' s brightness in relation to the brightness of the kernel of pixels around them. default is false. offset: The larger the offset value, the lower brightness pixels on the original image will be set to white. default is 0. invert: If true, the image will be inverted before the operation. default



item	description
	is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *gaussian(int size,
    | bool unsharp = false, float mul =
    | -1, float add = 0.0, bool
    | threshold = false, int offset =
    | 0, bool invert = false,
    | image::Image *mask = nullptr)
```

5.22.69. laplacian

```
1 | def laplacian(self, size: int,
    | sharpen: bool = False, mul: float =
    | -1, add: float = 0.0, threshold:
    | bool = False, offset: int = 0,
    | invert: bool = False, mask: Image =
    | None) -> Image
```

Convolves the image by a edge detecting laplacian kernel.

item	description
type	func
param	size: Kernel size. The actual kernel size is $((size * 2) + 1) * ((size * 2) + 1)$. Use 1(3x3 kernel), 2(5x5 kernel). sharpen: If True, this method will sharpen the image instead of an unthresholded edge detection image. Then increase the kernel size to improve image clarity. default is false. mul: This parameter is used to multiply the convolved pixel results. default is auto. add: This parameter is the value to be added to each convolution pixel result. default is 0.0. threshold: If true, which will enable adaptive thresholding of the image which sets pixels to white or black based on a pixel' s brightness in relation to the brightness of the kernel of pixels around them. default is false. offset: The larger the offset value, the lower brightness pixels on the original image will be set to white. default is 0. invert: If true, the image will be inverted before the operation. default

item	description
	is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *laplacian(int size,
    | bool sharpen = false, float mul =
    | -1, float add = 0.0, bool
    | threshold = false, int offset =
    | 0, bool invert = false,
    | image::Image *mask = nullptr)
```

5.22.70. bilateral

```
1 | def bilateral(self, size: int,
    | color_sigma: float = 0.1,
    | space_sigma: float = 1, threshold:
    | bool = False, offset: int = 0,
    | invert: bool = False, mask: Image =
    | None) -> Image
```

Convolve the image by a bilateral filter.

item	description
type	func
param	size: Kernel size. The actual kernel size is $((\text{size} * 2) + 1) * ((\text{size} * 2) + 1)$. Use 1(3x3 kernel), 2(5x5 kernel). color_sigma: Controls how closely colors are matched using the bilateral filter. default is 0.1. space_sigma: Controls how closely pixels space-wise are blurred with each other. default is 1. threshold: If true, which will enable adaptive thresholding of the image which sets pixels to white or black based on a pixel' s brightness in relation to the brightness of the kernel of pixels around them. default is false. offset: The larger the offset value, the lower brightness pixels on the original image will be set to white. default is 0. invert: If true, the image will be inverted before the operation. default is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on.



item	description
	Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```
1 | image::Image *bilateral(int size,
    | double color_sigma = 0.1, double
    | space_sigma = 1, bool threshold =
    | false, int offset = 0, bool
    | invert = false, image::Image
    | *mask = nullptr)
```

5.22.71. lnpolar

```
1 | def lnpolar(self, reverse: bool =
    | False) -> Image
```

Re-project' s and image from cartessian coordinates to linear polar coordinates.

item	description
type	func
param	reverse: If true, the image will be reverse polar transformed. default is false.
return	Returns the image after the operation is completed.

item	description
static	False
C++ definition code:	
<pre>1 image::Image *linpolar(bool reverse = false)</pre>	

5.22.72. logpolar

```
1 | def logpolar(self, reverse: bool =
   | False) -> Image
```

Re-project' s and image from cartessian coordinates to log polar coordinates.

item	description
type	func
param	reverse: If true, the image will be reverse polar transformed. default is false.
return	Returns the image after the operation is completed.
static	False
C++ definition code:	
<pre>1 image::Image *logpolar(bool reverse = false)</pre>	

5.22.73. lens_corr

```
1 | def lens_corr(self, strength: float  
   | = 1.8, zoom: float = 1.0, x_corr:  
   | float = 0.0, y_corr: float = 0.0) ->  
   | Image
```

Performs a lens correction operation on the image.

TODO: support in the feature

item	description
type	func
param	strength : The strength of the lens correction. default is 1.8. zoom : The zoom of the lens correction. default is 1.0. x_corr : The x correction of the lens correction. default is 0.0. y_corr : The y correction of the lens correction. default is 0.0.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```
1 | image::Image *lens_corr(double  
   | strength = 1.8, double zoom =  
   | 1.0, double x_corr = 0.0, double  
   | y_corr = 0.0)
```

5.22.74. rotation_corr

```

1 | def rotation_corr(self, x_rotation:
    float = 0.0, y_rotation: float =
    0.0, z_rotation: float = 0.0,
    x_translation: float = 0.0,
    y_translation: float = 0.0, zoom:
    float = 1.0, fov: float = 60.0,
    corners: list[float] = []) -> Image

```

Performs a rotation correction operation on the image. TODO: support in the feature

item	description
type	func
param	x_rotation: The x rotation of the rotation correction. default is 0.0. y_rotation: The y rotation of the rotation correction. default is 0.0. z_rotation: The z rotation of the rotation correction. default is 0.0. x_translation: The x translation of the rotation correction. default is 0.0. y_translation: The y translation of the rotation correction. default is 0.0. zoom: The zoom of the rotation correction. default is 1.0. fov: The fov of the rotation correction. default is 60.0. corners: The corners of the rotation correction. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image
   | *rotation_corr(double x_rotation
   | = 0.0, double y_rotation = 0.0,
   | double z_rotation = 0.0, double
   | x_translation = 0.0, double
   | y_translation = 0.0, double zoom
   | = 1.0, double fov = 60.0,
   | std::vector<float> corners =
   | std::vector<float>())
```

5.22.75. get_histogram

```
1 | def get_histogram(self, thresholds:
   | list[list[int]] = [], invert: bool =
   | False, roi: list[int] = [], bins:
   | int = -1, l_bins: int = 100, a_bins:
   | int = 256, b_bins: int = 256,
   | difference: Image = None) ->
   | Histogram
```

Computes the normalized histogram on all color channels and returns a image::Histogram object.

item	description
type	func
note	For GRAYSCALE format, Lmin and Lmax range is [0, 255]. For RGB888 format, Lmin and Lmax range is [0, 100].
param	thresholds: You can define multiple thresholds. For GRAYSCALE format, you can use

item	description
	{{Lmin, Lmax}, ...} to define one or more thresholds. For RGB888 format, you can use {{Lmin, Lmax, Amin, Amax, Bmin, Bmax}, ...} to define one or more thresholds. Where the upper case L,A,B represent the L,A,B channels of the LAB image format, and min, max represent the minimum and maximum values of the corresponding channels. invert: If true, the thresholds will be inverted before the operation. default is false. roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. bins: The number of bins to use for the histogram. In GRAYSCALE format, setting range is [2, 256], default is 100. In RGB888 format, setting range is [2, 100], default is 100. I_bins: The number of bins to use for the I channel of the histogram. Only valid in RGB888 format. If an invalid value is set, bins will be used instead. The setting range is [2,



item	description
	100], default is 100. a_bins: The number of bins to use for the a channel of the histogram. Only valid in RGB888 format. The setting range is [2, 256], default is 256. b_bins: The number of bins to use for the b channel of the histogram. Only valid in RGB888 format. The setting range is [2, 256], default is 256. difference: difference may be set to an image object to cause this method to operate on the difference image between the current image and the difference image object. default is None.
return	Returns image::Histogram object
static	False

C++ defination code:

```

image::Histogram
get_histogram(std::vector<std::vector<int>>(),
thresholds =
std::vector<std::vector<int>>(),
bool invert = false,
std::vector<int> roi =
std::vector<int>(), int bins =
-1, int l_bins = 100, int a_bins
= 256, int b_bins = 256,
image::Image *difference =

```

```
nullptr)
```

5.22.76. get_statistics

```
1 | def get_statistics(self, thresholds:  
  list[list[int]] = [], invert: bool =  
  False, roi: list[int] = [], bins:  
  int = -1, l_bins: int = -1, a_bins:  
  int = -1, b_bins: int = -1,  
  difference: Image = None) ->  
  Statistics
```

Gets the statistics of the image. TODO: support in the feature

item	description
type	func
note	For GRAYSCALE format, Lmin and Lmax range is [0, 255]. For RGB888 format, Lmin and Lmax range is [0, 100].
param	thresholds: You can define multiple thresholds. For GRAYSCALE format, you can use {{Lmin, Lmax}, ...} to define one or more thresholds. For RGB888 format, you can use {{Lmin, Lmax, Amin, Amax, Bmin, Bmax}, ...} to define one or more thresholds. Where the upper case L,A,B represent the L,A,B channels of the LAB image

item	description
	format, and min, max represent the minimum and maximum values of the corresponding channels. invert: If true, the image will be inverted before the operation. default is false. roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. bins: The number of bins to use for the statistics. default is -1. l_bins: The number of bins to use for the l channel of the statistics. default is -1. a_bins: The number of bins to use for the a channel of the statistics. default is -1. b_bins: The number of bins to use for the b channel of the statistics. default is -1. difference: The difference image to use for the statistics. default is None.
return	Returns the statistics of the image
static	False

C++ defination code:



```

1 | image::Statistics
   | get_statistics(std::vector<std::vect
   | thresholds =
   | std::vector<std::vector<int>>(),
   | bool invert = false,
   | std::vector<int> roi =
   | std::vector<int>(), int bins =
   | -1, int l_bins = -1, int a_bins =
   | -1, int b_bins = -1, image::Image
   | *difference = nullptr)

```

5.22.77. get_regression

```

1 | def get_regression(self, thresholds:
   | list[list[int]] = [], invert: bool =
   | False, roi: list[int] = [],
   | x_stride: int = 2, y_stride: int =
   | 1, area_threshold: int = 10,
   | pixels_threshold: int = 10, robust:
   | bool = False) -> list[Line]

```

Gets the regression of the image.

item	description
type	func
note	For GRAYSCALE format, Lmin and Lmax range is [0, 255]. For RGB888 format, Lmin and Lmax range is [0, 100].
param	thresholds: You can define multiple thresholds. For GRAYSCALE format, you can use {{Lmin, Lmax}, ...} to define one or more thresholds.

item	description
	For RGB888 format, you can use {{Lmin, Lmax, Amin, Amax, Bmin, Bmax}, ...} to define one or more thresholds. Where the upper case L,A,B represent the L,A,B channels of the LAB image format, and min, max represent the minimum and maximum values of the corresponding channels. invert: If true, the image will be inverted before the operation. default is false. roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. x_stride: The x stride to use for the regression. default is 2. y_stride: The y stride to use for the regression. default is 1. area_threshold: The area threshold to use for the regression. default is 10. pixels_threshold: The pixels threshold to use for the regression. default is 10. robust: If true, the regression will be robust. default is false.



item	description
return	Returns the regression of the image
static	False

C++ defination code:

```

1 | std::vector<image::Line>
   | get_regression(std::vector<std::vect
   | thresholds =
   | std::vector<std::vector<int>>(),
   | bool invert = false,
   | std::vector<int> roi =
   | std::vector<int>(), int x_stride
   | = 2, int y_stride = 1, int
   | area_threshold = 10, int
   | pixels_threshold = 10, bool
   | robust = false)

```

5.22.78. save

```

1 | def save(self, path: str, quality:
   | int = 95) -> maix.err.Err

```

Save image to file

item	description
type	func
param	path : file path quality : image quality, by default(value is 95), support jpeg and png format

item	description
return	error code, err::ERR_NONE is ok, other is error
static	False

C++ definition code:

```
1 | err::Err save(const char *path,
    | int quality = 95)
```

5.22.79. flood_fill

```
1 | def flood_fill(self, x: int, y: int,
    | seed_threshold: float = 0.05,
    | floating_threshold: float = 0.05,
    | color: Color = ..., invert: bool =
    | False, clear_background: bool =
    | False, mask: Image = None) -> Image
```

Flood fills a region of the image starting from location x, y.

item	description
type	func
param	x: The x coordinate of the seed point. y: The y coordinate of the seed point. seed_threshold: The seed_threshold value controls how different any pixel in the fill area may be from the original starting pixel. default is 0.05. floating_threshold: The floating_threshold value controls how

item	description
	different any pixel in the fill area may be from any neighbor pixels. default is 0.05. color: The color to fill the region with. default is white. invert: If true, the image will be inverted before the operation. default is false. clear_background: If true, the background will be cleared before the operation. default is false. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None. FIXME: the mask image works abnormally
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```

1 | image::Image *flood_fill(int x,
   | int y, float seed_threshold =
   | 0.05, float floating_threshold =
   | 0.05, image::Color color =
   | image::COLOR_WHITE, bool invert =
   | false, bool clear_background =

```

```
false, image::Image *mask =
nullptr)
```

5.22.80. erode

```
1 | def erode(self, size: int,
    threshold: int = -1, mask: Image =
    None) -> Image
```

Erodes the image in place.

item	description
type	func
param	size: Kernel size. The actual kernel size is $((size * 2) + 1) * ((size * 2) + 1)$. Use 1(3x3 kernel), 2(5x5 kernel). threshold: The number of pixels in the kernel that are not 0. If it is less than or equal to the threshold, set the center pixel to black. default is $(kernel_size - 1)$. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.

item	description
static	False
C++ definition code:	
<pre>1 image::Image *erode(int size, int threshold = -1, image::Image *mask = nullptr)</pre>	

5.22.81. dilate

```
1 | def dilate(self, size: int,
    | threshold: int = 0, mask: Image =
    | None) -> Image
```

Dilates the image in place.

item	description
type	func
param	size: Kernel size. The actual kernel size is $((size * 2) + 1) * ((size * 2) + 1)$. Use 1(3x3 kernel), 2(5x5 kernel). threshold: The number of pixels in the kernel that are not 0. If it is greater than or equal to the threshold, set the center pixel to white. default is 0. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the

item	description
	image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ definition code:

```
1 | image::Image *dilate(int size,
    | int threshold = 0, image::Image
    | *mask = nullptr)
```

5.22.82. open

```
1 | def open(self, size: int, threshold:
    | int = 0, mask: Image = None) ->
    | Image
```

Performs erosion and dilation on an image in order.

item	description
type	func
param	size: Kernel size. The actual kernel size is $((size * 2) + 1) * ((size * 2) + 1)$. Use 1(3x3 kernel), 2(5x5 kernel). threshold: As the threshold for erosion and dilation, the actual threshold for erosion is $(kernel_size - 1 - threshold)$, the actual threshold

item	description
	for dialation is threshold. default is 0. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```
1 | image::Image *open(int size, int
   | threshold = 0, image::Image *mask
   | = nullptr)
```

5.22.83. close

```
1 | def close(self, size: int,
   | threshold: int = 0, mask: Image =
   | None) -> Image
```

Performs dilation and erosion on an image in order.

item	description
type	func

item	description
param	size: Kernel size. The actual kernel size is $((\text{size} * 2) + 1) * ((\text{size} * 2) + 1)$. Use 1(3x3 kernel), 2(5x5 kernel). threshold: As the threshold for erosion and dilation, the actual threshold for erosion is $(\text{kernel_size} - 1 - \text{threshold})$, the actual threshold for dialation is threshold. default is 0. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```
1 | image::Image *close(int size, int
   | threshold = 0, image::Image *mask
   | = nullptr)
```

5.22.84. top_hat

```
1 | def top_hat(self, size: int,
   | threshold: int = 0, mask: Image =
```

```
None) -> Image
```

Returns the image difference of the image and Image.open()' ed image.

item	description
type	func
param	size: Kernel size. The actual kernel size is $((size * 2) + 1) * ((size * 2) + 1)$. Use 1(3x3 kernel), 2(5x5 kernel). threshold: As the threshold for open method. default is 0. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```
1 | image::Image *top_hat(int size,  
    | int threshold = 0, image::Image  
    | *mask = nullptr)
```

5.22.85. black_hat

```
1 | def black_hat(self, size: int,
    threshold: int = 0, mask: Image =
    None) -> Image
```

Returns the image difference of the image and Image.close()' ed image.

item	description
type	func
param	size: Kernel size. The actual kernel size is $((size * 2) + 1) * ((size * 2) + 1)$. Use 1(3x3 kernel), 2(5x5 kernel). threshold: As the threshold for close method. default is 0. mask: Mask is another image to use as a pixel level mask for the operation. The mask should be an image with just black or white pixels and should be the same size as the image being operated on. Only pixels set in the mask are modified. default is None.
return	Returns the image after the operation is completed.
static	False

C++ defination code:

```
1 | image::Image *black_hat(int size,
    int threshold = 0, image::Image
    *mask = nullptr)
```

5.22.86. find_blobs

```
1 | def find_blobs(self, thresholds:  
    list[list[int]] = [], invert: bool =  
    False, roi: list[int] = [],  
    x_stride: int = 2, y_stride: int =  
    1, area_threshold: int = 10,  
    pixels_threshold: int = 10, merge:  
    bool = False, margin: int = 0,  
    x_hist_bins_max: int = 0,  
    y_hist_bins_max: int = 0) ->  
    list[Blob]
```

Finds all blobs in the image and returns a list of image.Blob class which describe each Blob.\nPlease see the image.Blob object more more information.

item	description
type	func
note	For GRAYSCALE format, Lmin and Lmax range is [0, 255]. For RGB888 format, Lmin and Lmax range is [0, 100].
param	thresholds: You can define multiple thresholds. For GRAYSCALE format, you can use {{Lmin, Lmax}, ...} to define one or more thresholds. For RGB888 format, you can use {{Lmin, Lmax, Amin, Amax, Bmin, Bmax}, ...} to define one or more thresholds. Where the upper case L,A,B represent

item	description
	the L,A,B channels of the LAB image format, and min, max represent the minimum and maximum values of the corresponding channels. invert: if true, will invert thresholds before find blobs, default is false roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. x_stride: x stride is the number of x pixels to skip when doing the hough transform. default is 2 y_stride: y_stride is the number of y pixels to skip when doing the hough transform. default is 1 area_threshold: area threshold, if the blob area is smaller than area_threshold, the blob is not returned, default is 10 pixels_threshold: pixels threshold, if the blob pixels is smaller than area_threshold, the blob is not returned,, default is 10. when x_stride and y_stride is equal to 1, pixels_threshold is equivalent to area_threshold merge: if True merges all not filtered out blobs whos bounding rectangles



item	description
	intersect each other. default is false margin: margin can be used to increase or decrease the size of the bounding rectangles for blobs during the intersection test. For example, with a margin of 1 blobs whos bounding rectangles are 1 pixel away from each other will be merged. default is 0 x_hist_bins_max: if set to non-zero populates a histogram buffer in each blob object with an x_histogram projection of all columns in the object. This value then sets the number of bins for that projection. y_hist_bins_max: if set to non-zero populates a histogram buffer in each blob object with an y_histogram projection of all rows in the object. This value then sets the number of bins for that projection.
return	Return the blob when found blobs, format is (blob1, blob2, ...), you can use blob class methods to do more operations.
static	False

C++ defination code:

```
std::vector<image::Blob>
find_blobs(std::vector<std::vector<
```

```

thresholds =
std::vector<std::vector<int>>>(),
bool invert = false,
std::vector<int> roi =
std::vector<int>(), int x_stride
= 2, int y_stride = 1, int
area_threshold = 10, int
pixels_threshold = 10, bool merge
= false, int margin = 0, int
x_hist_bins_max = 0, int
y_hist_bins_max = 0)

```

5.22.87. find_lines

```

1 | def find_lines(self, roi: list[int]
   | = [], x_stride: int = 2, y_stride:
   | int = 1, threshold: float = 1000,
   | theta_margin: float = 25,
   | rho_margin: float = 25) ->
   | list[Line]

```

Find lines in image

item	description
type	func
param	roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. x_stride: x stride is the number of x pixels to skip when doing the hough transform. default is 2 y_stride: y_stride is the number of y

item	description
	pixels to skip when doing the hough transform. default is 1 threshold: threshold threshold controls what lines are detected from the hough transform. Only lines with a magnitude greater than or equal to threshold are returned. The right value of threshold for your application is image dependent. default is 1000. theta_margin: theta_margin controls the merging of detected lines. default is 25. rho_margin: rho_margin controls the merging of detected lines. default is 25.
return	Return the line when found lines, format is (line1, line2, ...), you can use line class methods to do more operations
static	False

C++ defination code:

```

1 | std::vector<image::Line>
   | find_lines(std::vector<int> roi =
   | std::vector<int>(), int x_stride
   | = 2, int y_stride = 1, double
   | threshold = 1000, double
   | theta_margin = 25, double
   | rho_margin = 25)

```

5.22.88. find_line_segments

```
1 | def find_line_segments(self, roi:  
    list[int] = [], merge_distance: int  
    = 0, max_theta_difference: int = 15)  
    -> list[Line]
```

Finds all line segments in the image.

item	description
type	func
param	roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. merge_distance: The maximum distance between two lines to merge them. default is 0. max_theta_difference: The maximum difference between two lines to merge them. default is 15.
return	Return the line when found lines, format is (line1, line2, ...), you can use line class methods to do more operations
static	False

C++ defination code:

```

1 | std::vector<image::Line>
   find_line_segments(std::vector<int>
   roi = std::vector<int>(), int
   merge_distance = 0, int
   max_theta_difference = 15)

```

5.22.89. search_line_path

```

1 | def search_line_path(self,
   threshold: int = 30, merge_degree:
   int = 10, min_len_of_new_path: int =
   10) -> list[LineGroup]

```

Search the path of line

item	description
type	func
param	threshold: Threshold for finding a line, the larger the value the more accurate the line is found merge_degree: Minimum angle difference required when merging multiple lines min_len_of_new_path: The minimum length of a new path, if the crossing length exceeds this value, it is considered a new path.
return	Return the line when found lines, format is (groupline1, groupline2, ...), you can use LineGroup class methods to do more operations

item	description
static	False
C++ defination code:	
<pre> 1 std::vector<image::LineGroup> search_line_path(int threshold = 30, int merge_degree = 10, int min_len_of_new_path = 10) </pre>	

5.22.90. find_circles

```

1 | def find_circles(self, roi:
   | list[int] = [], x_stride: int = 2,
   | y_stride: int = 1, threshold: int =
   | 2000, x_margin: int = 10, y_margin:
   | int = 10, r_margin: int = 10, r_min:
   | int = 2, r_max: int = -1, r_step:
   | int = 2) -> list[Circle]

```

Find circles in image

item	description
type	func
param	roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. x_stride: x stride is the number of x pixels to skip when doing the hough transform. default is 2

item	description
	y_stride: y_stride is the number of y pixels to skip when doing the hough transform. default is 1 threshold: threshold controls what circles are detected from the hough transform. Only circles with a magnitude greater than or equal to threshold are returned. The right value of threshold for your application is image dependent. x_margin: x_margin controls the merging of detected circles. Circles which are x_margin, y_margin, and r_margin pixels apart are merged. default is 10 y_margin: y_margin controls the merging of detected circles. Circles which are x_margin, y_margin, and r_margin pixels apart are merged. default is 10 r_margin: r_margin controls the merging of detected circles. Circles which are x_margin, y_margin, and r_margin pixels apart are merged. default is 10 r_min: r_min controls the minimum circle radius detected. Increase this to speed up the algorithm. default is 2 r_max: r_max controls the maximum circle radius detected. Decrease this to speed up the algorithm. default is



item	description
	$\min(\text{roi.w} / 2, \text{roi.h} / 2)$ r_step : r_step controls how to step the radius detection by. default is 2.
return	Return the circle when found circles, format is (circle1, circle2, ...), you can use circle class methods to do more operations
static	False

C++ defination code:

```
1 | std::vector<image::Circle>
   | find_circles(std::vector<int> roi
   | = std::vector<int>(), int
   | x_stride = 2, int y_stride = 1,
   | int threshold = 2000, int
   | x_margin = 10, int y_margin = 10,
   | int r_margin = 10, int r_min = 2,
   | int r_max = -1, int r_step = 2)
```

5.22.91. find_rects

```
1 | def find_rects(self, roi: list[int]
   | = [], threshold: int = 10000) ->
   | list[Rect]
```

Finds all rects in the image.

item	description
type	func

item	description
param	roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. threshold: The threshold to use for the rects. default is 10000.
return	Returns the rects of the image
static	False

C++ definition code:

```
1 | std::vector<image::Rect>
   | find_rects(std::vector<int> roi =
   | std::vector<int>(), int threshold
   | = 10000)
```

5.22.92. find_qrcodes

```
1 | def find_qrcodes(self, roi:
   | list[int] = [], decoder_type:
   | QRCodeDecoderType = ...) ->
   | list[QRCode]
```

Finds all qrcodes in the image.

item	description
type	func

item	description
param	roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. decoder_type: Select the QR code decoding method. Choosing QRCODE_DECODER_TYPE_QUIRC allows for retrieving QR code version, ECC level, mask, data type, and other details, though it may decode slower at lower resolutions. Opting for QRCODE_DECODER_TYPE_ZBAR enables faster decoding at lower resolutions but may slow down at higher resolutions, providing only the QR code content and position information. default is QRCODE_DECODER_TYPE_ZBAR.
return	Returns the qrcodes of the image
static	False

C++ defination code:

```
std::vector<image::QRCode>
find_qrcodes(std::vector<int> roi
= std::vector<int>(),
image::QRCodeDecoderType
decoder_type =
```

5.22.93. find_apriltags

```
1 | def find_apriltags(self, roi:  
    list[int] = [], families:  
    ApriltagFamilies = ..., fx: float =  
    -1, fy: float = -1, cx: int = -1,  
    cy: int = -1) -> list[AprilTag]
```

Finds all apriltags in the image.

item	description
type	func
param	roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. families: The families to use for the apriltags. default is TAG36H11. fx: The camera X focal length in pixels, default is -1. fy: The camera Y focal length in pixels, default is -1. cx: The camera X center in pixels, default is image.width / 2. cy: The camera Y center in pixels, default is image.height / 2.
return	Returns the apriltags of the image

item	description
static	False

C++ definition code:

```
1 | std::vector<image::AprilTag>
   | find_apriltags(std::vector<int>
   | roi = std::vector<int>(),
   | image::ApriltagFamilies families
   | =
   | image::ApriltagFamilies::TAG36H11,
   | float fx = -1, float fy = -1, int
   | cx = -1, int cy = -1)
```

5.22.94. find_datamatrices

```
1 | def find_datamatrices(self, roi:
   | list[int] = [], effort: int = 200) -
   | > list[DataMatrix]
```

Finds all datamatrices in the image.

item	description
type	func

item	description
param	roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. effort: Controls how much time to spend trying to find data matrix matches. default is 200.
return	Returns the datamatrices of the image
static	False

C++ definition code:

```
1 | std::vector<image::DataMatrix>
   | find_datamatrices(std::vector<int>
   | roi = std::vector<int>(), int
   | effort = 200)
```

5.22.95. find_barcodes

```
1 | def find_barcodes(self, roi:
   | list[int] = []) -> list[BarCode]
```

Finds all barcodes in the image.

item	description
type	func

item	description
param	roi : The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image.
return	Returns the barcodes of the image
static	False

C++ definition code:

```
1 | std::vector<image::BarCode>
   | find_barcodes(std::vector<int>
   | roi = std::vector<int>())
```

5.22.96. find_displacement

```
1 | def find_displacement(self,
   | template_image: Image, roi:
   | list[int] = [], template_roi:
   | list[int] = [], logpolar: bool =
   | False) -> Displacement
```

Finds the displacement between the image and the template. TODO: support in the feature
 note: this method must be used on power-of-2 image sizes

item	description
type	func

item	description
param	template_image: The template image. roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. template_roi: The region-of-interest rectangle (x, y, w, h) to work in. If not specified, it is equal to the image rectangle. logpolar: If true, it will instead find rotation and scale changes between the two images. default is false.
return	Returns the displacement of the image
static	False

C++ definition code:

```

1 | image::Displacement
   | find_displacement(image::Image
   | &template_image, std::vector<int>
   | roi = std::vector<int>(),
   | std::vector<int> template_roi =
   | std::vector<int>(), bool logpolar
   | = false)

```

5.22.97. find_template

```

1 | def find_template(self,
   | template_image: Image, threshold:
   | float, roi: list[int] = [], step:
   | int = 2, search: TemplateMatch =
   | ...) -> list[int]

```

Finds the template in the image.

item	description
type	func
param	template_image: The template image. threshold: Threshold is floating point number (0.0-1.0) where a higher threshold prevents false positives while lowering the detection rate while a lower threshold does the opposite. roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. Only valid in SEARCH_EX mode. step: The step size to use for the template. default is 2. Only valid in SEARCH_EX mode search: The search method to use for the template. default is SEARCH_EX.
return	Returns a bounding box tuple (x, y, w, h) for the matching location

item	description
	otherwise None.
static	False

C++ defination code:

```

1 | std::vector<int>
   | find_template(image::Image
   | &template_image, float threshold,
   | std::vector<int> roi =
   | std::vector<int>(), int step = 2,
   | image::TemplateMatch search =
   | image::TemplateMatch::SEARCH_EX)

```

5.22.98. find_features

```

1 | def find_features(self, cascade:
   | int, threshold: float = 0.5, scale:
   | float = 1.5, roi: list[int] = []) ->
   | list[int]

```

Finds the features in the image. TODO: support in the feature

item	description
type	func
param	cascade: The cascade to use for the features. default is CASCADE_FRONTALFACE_ALT. threshold: The threshold to use for the features. default is 0.5. scale: The scale to use for the features. default is 1.5.

item	description
	roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image.
return	Returns the features of the image
static	False

C++ defination code:

```
1 | std::vector<int>
   | find_features(int cascade, float
   | threshold = 0.5, float scale =
   | 1.5, std::vector<int> roi =
   | std::vector<int>())
```

5.22.99. find_lbp

```
1 | def find_lbp(self, roi: list[int] =
   | []) -> LBPKeyPoint
```

Finds the lbp in the image. TODO: support in the feature.

item	description
type	func
param	roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left

item	description
	corner, w and h are the width and height of roi. default is None, means whole image.
return	Returns the lbp of the image
static	False

C++ definition code:

```
1 | image::LBPKeyPoint
   | find_lbp(std::vector<int> roi =
   | std::vector<int>())
```

5.22.100. find_keypoints

```
1 | def find_keypoints(self, roi:
   | list[int] = [], threshold: int = 20,
   | normalized: bool = False,
   | scale_factor: float = 1.5,
   | max_keypoints: int = 100,
   | corner_detector: CornerDetector =
   | ...) -> ORBKeyPoint
```

Finds the keypoints in the image. TODO: support in the feature.

item	description
type	func
param	roi : The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and

item	description
	height of roi. default is None, means whole image. threshold: The threshold to use for the keypoints. default is 20. normalized: If true, the image will be normalized before the operation. default is false. scale_factor: The scale factor to use for the keypoints. default is 1.5. max_keypoints: The maximum number of keypoints to use for the keypoints. default is 100. corner_detector: The corner detector to use for the keypoints. default is CORNER_AGAST.
return	Returns the keypoints of the image
static	False

C++ defination code:

```

1 | image::ORBKeyPoint
   | find_keypoints(std::vector<int>
   | roi = std::vector<int>(), int
   | threshold = 20, bool normalized =
   | false, float scale_factor = 1.5,
   | int max_keypoints = 100,
   | image::CornerDetector
   | corner_detector =
   | image::CornerDetector::CORNER_AGAST

```

5.22.101. find_edges

```

1 | def find_edges(self, edge_type:
    EdgeDetector, roi: list[int] = [],
    threshold: list[int] = [100, 200]) -
    > Image

```

Finds the edges in the image.

item	description
type	func
param	edge_type: The edge type to use for the edges. default is EDGE_CANNY. roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. threshold: The threshold to use for the edges. default is 20.
return	Returns the edges of the image
static	False

C++ defination code:

```

1 | image::Image*
    find_edges(image::EdgeDetector
    edge_type, std::vector<int> roi =
    std::vector<int>(),
    std::vector<int> threshold =
    std::vector<int>({100, 200}))

```

5.22.102. find_hog

```
1 | def find_hog(self, roi: list[int] =
    [], size: int = 8) -> Image
```

Finds the hog in the image. TODO: support in the feature

item	description
type	func
param	roi: The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left corner, w and h are the width and height of roi. default is None, means whole image. size: The size to use for the hog. default is 8.
return	Returns the hog of the image
static	False

C++ defination code:

```
1 | image::Image*
    find_hog(std::vector<int> roi =
    std::vector<int>(), int size = 8)
```

5.22.103. match_lbp_descriptor

```
1 | def match_lbp_descriptor(self,
    desc1: LBPKeyPoint, desc2:
    LBPKeyPoint) -> int
```

Matches the lbp descriptor of the image. TODO:
support in the feature

item	description
type	func
param	desc1 : The descriptor to use for the match. desc2 : The descriptor to use for the match.
return	Returns the match of the image
static	False

C++ definition code:

```
1 | int  
   | match_lbp_descriptor(image::LBPPKeyPo  
   | &desc1, image::LBPPKeyPoint  
   | &desc2)
```

5.22.104. match_orb_descriptor

```
1 | def match_orb_descriptor(self,  
   | desc1: ORBKeyPoint, desc2:  
   | ORBKeyPoint, threshold: int = 95,  
   | filter_outliers: bool = False) ->  
   | KPTMatch
```

Matches the orb descriptor of the image. TODO:
support in the feature

item	description
type	func
param	desc1: The descriptor to use for the match. desc2: The descriptor to use for the match. threshold: The threshold to use for the match. default is 95. filter_outliers: If true, the image will be filter_outliers before the operation. default is false.
return	Returns the match of the image
static	False

C++ definition code:

```
1 | image::KPTMatch
   | match_orb_descriptor(image::ORBKeyPo
   | &desc1, image::ORBKeyPoint
   | &desc2, int threshold = 95, bool
   | filter_outliers = false)
```

5.22.105. resize_map_pos

```
1 | def resize_map_pos(self, w_out: int,
   | h_out: int, fit: Fit, x: int, y:
   | int, w: int = -1, h: int = -1) ->
   | list[int]
```

map point position or rectangle position from this image size to another image size(resize)

item	description
type	func
param	int : h_out target image height fit : resize method, see <code>maix.image.Fit</code> x : original point x, or rectagle left-top point's x y : original point y, or rectagle left-top point's y w : original rectagle width, can be -1 if not use this arg, default -1. h : original rectagle height, can be -1 if not use this arg, default -1.
return	list type, [x, y] if map point, [x, y, w, h] if resize rectangle.
static	False

C++ defination code:

```
1 | std::vector<int>
   | resize_map_pos(int w_out, int
   | h_out, image::Fit fit, int x, int
   | y, int w = -1, int h = -1)
```

5.23. QRCodeDetector

QRCodeDetector class

C++ defination code:

```
1 | class QRCodeDetector
```

5.23.1. `__init__`

```
1 | def __init__(self) -> None
```

QRCodeDetector constructor.\nUse npu to accelerate the speed of QR code scanning, note that this object will occupy npu resources

item	description
type	func
static	False

C++ definition code:

```
1 | QRCodeDetector()
```

5.23.2. `detect`

```
1 | def detect(self, img: Image, roi:  
    list[int] = [], decoder_type:  
    QRCodeDecoderType = ...) ->  
    list[QRCode]
```

Finds all qrcodes in the image.

item	description
type	func
param	img : The image to find qrcodes. roi : The region of interest, input in the format of (x, y, w, h), x and y are the coordinates of the upper left

item	description
	corner, w and h are the width and height of roi. default is None, means whole image. decoder_type: Select the QR code decoding method. Choosing <code>QRCODE_DECODER_TYPE_QUIRC</code> allows for retrieving QR code version, ECC level, mask, data type, and other details, though it may decode slower at lower resolutions. Opting for <code>QRCODE_DECODER_TYPE_ZBAR</code> enables faster decoding at lower resolutions but may slow down at higher resolutions, providing only the QR code content and position information. default is <code>QRCODE_DECODER_TYPE_ZBAR</code>.
return	Returns the qrcodes of the image
static	False

C++ defination code:

```

1 | std::vector<image::QRCode>
   | detect(image::Image *img,
   | std::vector<int> roi =
   | std::vector<int>(),
   | image::QRCodeDecoderType
   | decoder_type =
   | image::QRCodeDecoderType::QRCODE_DEC

```